

TOPST 단기 교육 과정

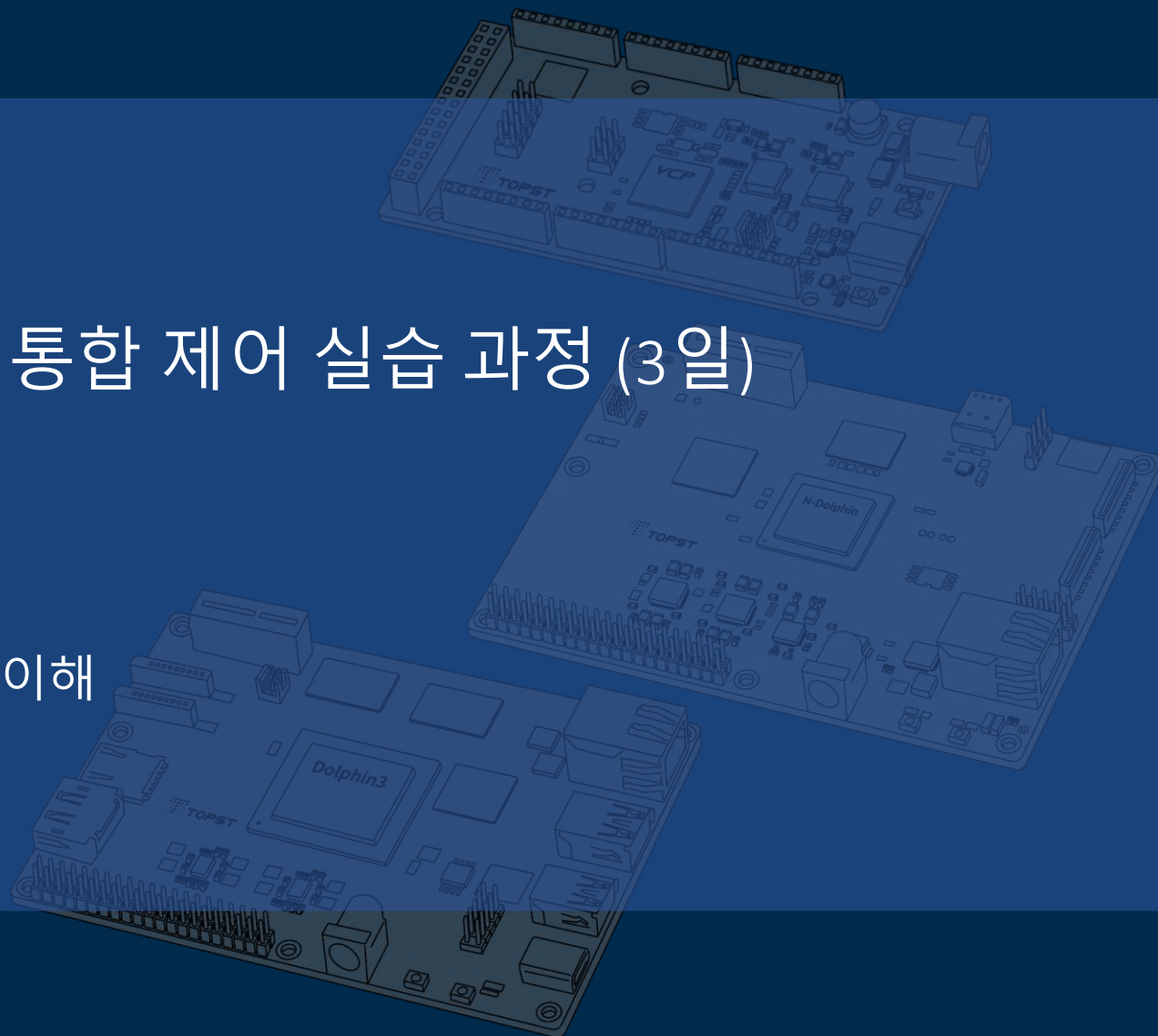
TOPST 임베디드·AI 통합 제어 실습 과정 (3일)



1일차

RTOS 기반 제어 구조 이해

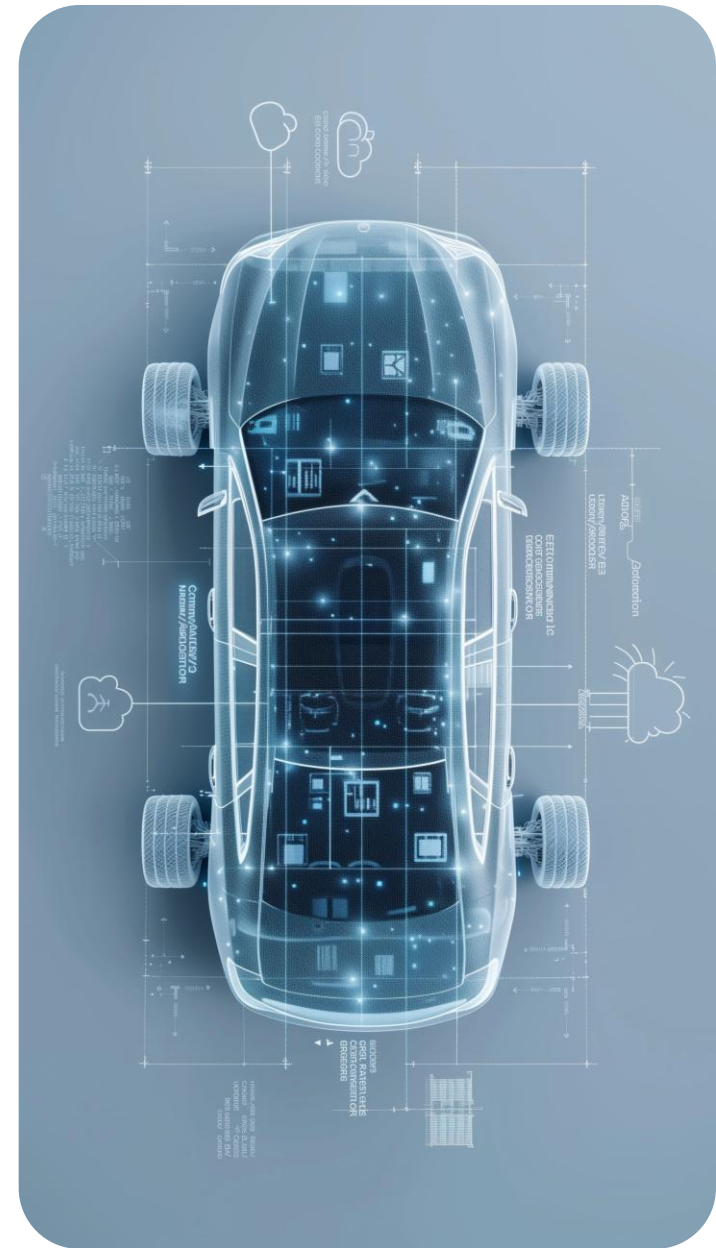
Telechips  **TOPST**



목차

Table of Contents

TOPST 제품 및 실습 과정 소개	01
RTOS 개요	02
VCP-G 개발 환경 설정	03
GPIO / ADC 기반 센서 제어	04
PWM / I2C 기반 센서 제어	05





TOPST

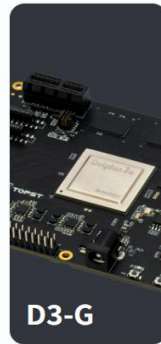
PRODUCT

TOPST offers full-featured Single Board Computers for development and production, alongside compact modules for embedded integration.

Development Boards

G Model

Powered by a field-proven SoC designed for mission-critical applications such as automotive, the G series ensures stable performance for diverse industrial, educational, and AI development needs.



D3-G



AI-G



VCP-G

Modular SoC Products

SoM Model

A compact, ready-to-integrate computing module that combines a processor, memory, and essential interfaces, designed to accelerate product development.

* SoM : (System on Module)



D5-CM

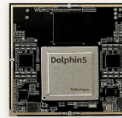
D5-CM is a compact System on Module integrating CPU, GPU, and 8TOPS-class NPU, ideal for rapid AI product development. Compatible with standard baseboards or custom carrier boards

Coming Soon ↗

SiP Model

A highly integrated chip package that combines key system components such as CPU, memory, and input/output interfaces in a compact form factor to deliver space-efficient performance. This product is intended for use in industrial solutions and will be available exclusively through professional and enterprise-level channels.

* SiP : (System in Package)



D5-SiP

AI-ready module with integrated CPU and 8TOPS NPU - optimized for edge AI and CPU processing.

Coming Soon ↗

<https://topst.ai>

<https://github.com/topst-development/Education>

Available Accessories

Industrial-Grade Accessories

This section includes both TOPST-developed accessories and verified third-party modules. All listed items are compatible with our hardware and tested for industrial and automotive use.



GSML-MIPI I/O Board

GSML-MIPI bidirectional sub-board with support for multiple virtual channels and Fakra connectors

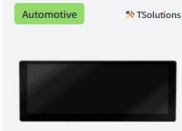
View Details



CAN Transceiver

Industrial-grade CAN interface board for real-time communication in embedded control systems

View Details



12.3" Display Board

Full-HD LVDS multi-touch panel with wide viewing angles, ideal for industrial and in-vehicle HMI

View Details



5MP MIPI CSI Camera

SMP image sensor with MIPI CSI output, suitable for AI vision tasks and camera integration on the TOPST boards.

View Details

Education Kits with Solution Partner

Academic Training Kit

While not developed by TOPST, these kits integrate our boards to provide practical, classroom-ready solutions for embedded and AI education.



D3-G EDU KIT

Learn core SoC architecture and high-performance processing using the D3-G board. This kit enables hands-on training in multicore computing, interface design, and automotive system control through realworldAP use cases like image processing and autonomous driving logic.

View Details



AI-G EDU KIT

Training kit built on the AI-G board, focusing on deep learning inference and parallel processing. Provides practical experience in vision AI and NPU-based decisionmaking, essential for autonomous driving applications.

View Details



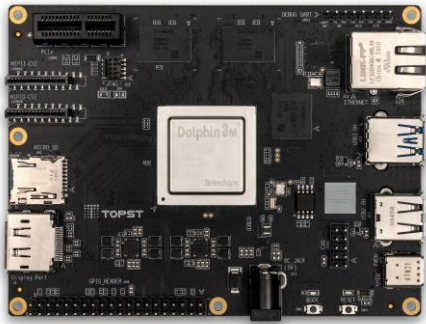
VCP-G EDU KIT

Hands-on kit for embedded system design using the VCP-G board. Covers FreeRTOS operations, realtime I/O control, and core vehicle ECU functions such as sensor integration and motor control in automotive electronics

View Details

TOPST 제품 및 실습 과정 소개

Description



다양한 개발을 지원하는 AP 기반 고성능 SBC

4K 디스플레이 3대를 지원하고, 여러 카메라 입력 및 실시간 2D/3D 그래픽 처리를 제공하여 산업 자동화, 스마트 키오스크, 스마트 홈 허브에 이상적인 제품입니다. 하드웨어 가상화를 통해 Hypervisor 없이도 두 개의 완벽히 분리시킨 상태에서 동시에 구동할 수 있어, 의료 기기, 보안 시스템, 고급 산업 제어 장치 등에서 뛰어난 성능을 발휘합니다.

Feature

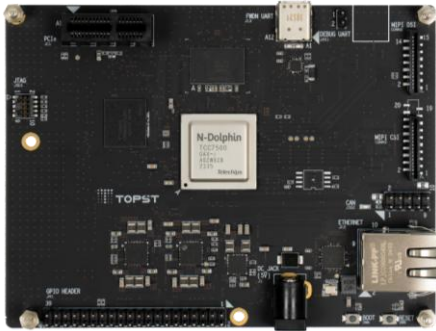
- **멀티 디스플레이 & 그래픽**
고화질 출력 및 2D/3D 프로세싱 지원
- **다채널 카메라 지원**
다중 카메라 입력 환경 지원을 위한 MIPI CSI 인터페이스
- **Hypervisor-less 솔루션**
가상화 장치 없이 다중 OS 지원
- **다양한 연결 지원**
USB 3.0, PCIe, Ethernet, 40 GPIO Pin, DP 지원
- **효율적인 프로세싱 구성**
멀티 태스킹을 위한 Cortex-A72 쿼드코어

Application

- **산업 자동화**
스마트 HMI, 로봇틱스, 다채널 카메라 분석환경
- **스마트 키오스크**
상호작용 키오스크, 디지털 메뉴 보드
- **물류**
AI 기반 디스플레이, 고객 행동분석 장비
- **보안**
감시카메라 허브, 안면인식 장비
- **스마트 홈**
멀티미디어 허브, 홈 보안 시스템

TOPST 제품 및 실습 과정 소개

Description



산업 전반에 걸쳐 사용 가능한 NPU 기반 엣지 비전 솔루션

첨단 비전 애플리케이션에 최적화된 고성능 NPU 기반 Edge AI 보드입니다.

ASIL-B 인증과 멀티채널 ISP 지원을 통해 뛰어난 이미지 품질과 견고한 엣지 처리 성능을 제공하며 산업 자동화, 스마트 감시 시스템, 헬스케어 분야에 적합합니다.

Feature

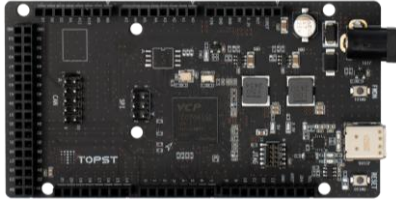
- **AI 프로세싱**
Edge 비전 응용을 위한 8 TOPS NPU
- **고속 카메라 입력**
다채널 이미지 캡처를 위한 MIPI CSI 2 인터페이스
- **고성능 출력**
MIPI DSI 및 내장 ISP 기반 FHD 디스플레이
- **퍼포먼스 효율**
실시간 처리를 위한 Cortex-A53 쿼드코어 (1.8GHz)
- **다양한 연결 지원**
PCIe, Ethernet, 40 GPIO Pin, DP 지원

Application

- **산업 자동화**
비전 기반 품질 검사, 로봇틱스
- **스마트 보안장비**
AI 기반 보안 카메라, 다채널 시스템
- **헬스케어**
이미지 분석, 환자 모니터링 시스템
- **리테일 및 물류**
자동 체크아웃 시스템, 패키지 검사 장비

TOPST 제품 및 실습 과정 소개

Description



실시간 Core 기반 다양한 산업의 변화를 이끌 Cortex-R 기반 MCU

저전력, 안전 및 실시간 처리 환경에 최적화된 32비트 MCU입니다. 자동차 분야에서 시작해, 현재는 산업 자동화, 스마트 홈, 보안 시스템 등에서도 뛰어난 성능을 발휘하고 있습니다. 텔레매틱스, 클러스터 관리 기능을 갖추고 있으며, 저전력 28nm 설계, 강력한 보안(EVITA Full HSM), ASIL-B 인증을 통해 안정적인 운영을 보장합니다.

Feature

- **저전력 고성능 MCU**
Cortex-R5F 기반 32-bit MCU (4MB Flash and 512KB SRAM)
- **보안 기능**
데이터 프로세싱 보안을 위한 EVITA Full HSM
- **ASIL B 수준의 실시간 동작 성능**
텔레매틱스, 클러스터 매니지먼트
- **인터페이스 구성**
CAN, SPI, 12-bit ADC 지원

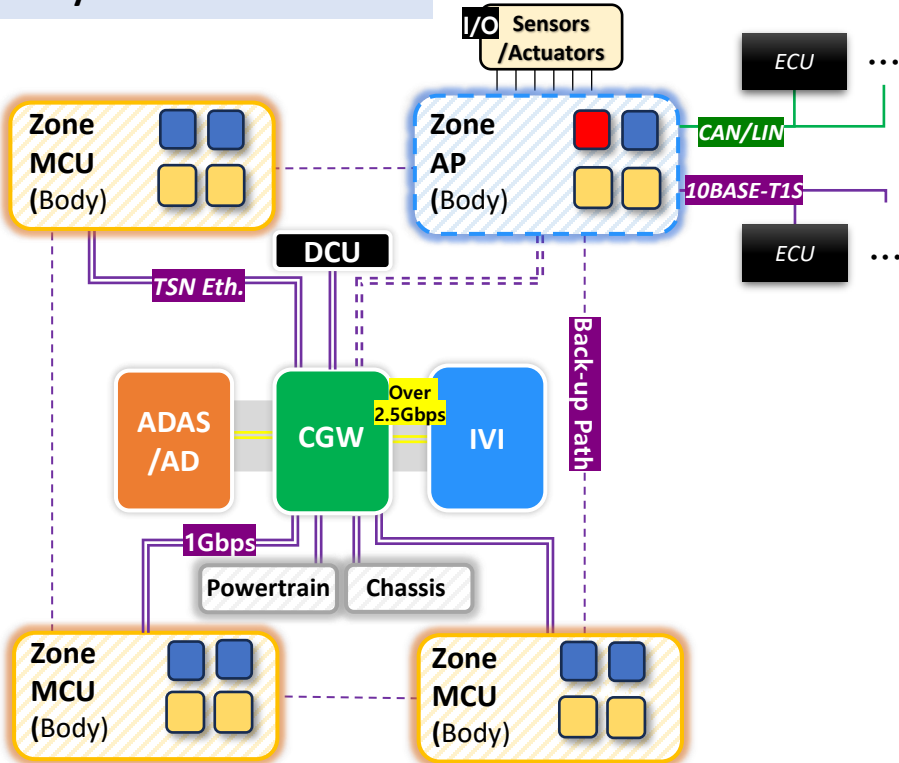
Application

- **산업 자동화**
실시간 모니터링, M2M 커뮤니케이션 보안
- **스마트 홈**
중앙 허브, 무선 충전 디바이스
- **보안 시스템**
접근 권한 관리, 보안카메라 허브
- **소비자 전자장치**
AI 기반 스마트 카메라 (제어)

E/E Architecture

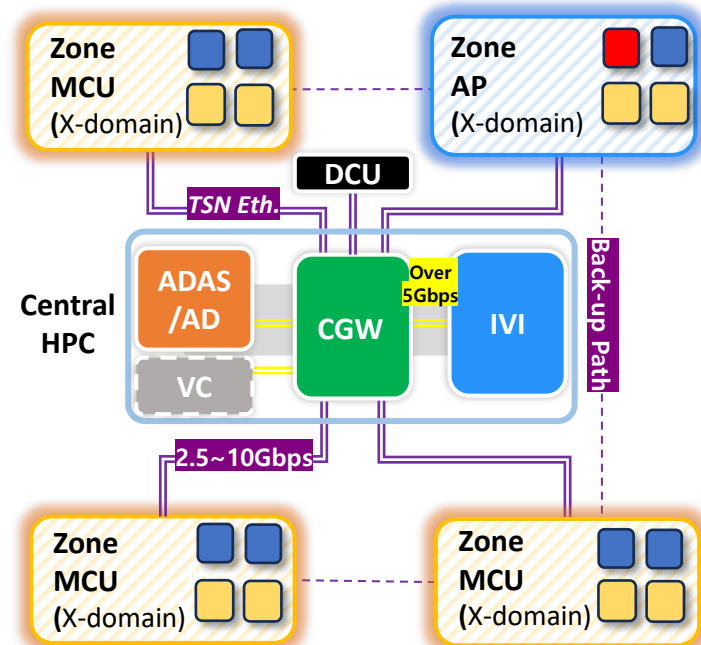
(Electrical/Electronic)

Body Zonal Architecture



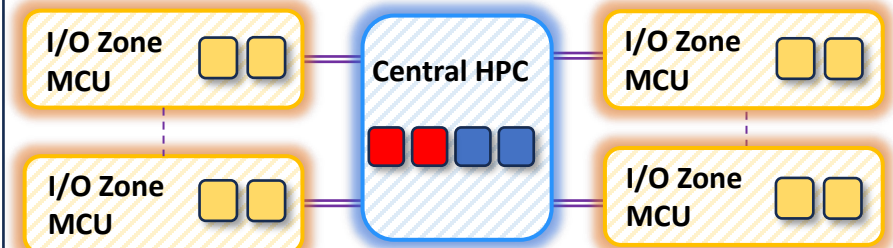
- : [MCU] Realtime Computation
- : [AP] Complex/High performance Computation
- : Aggregated I/O Control & G/W

Cross-Domain Zonal Architecture



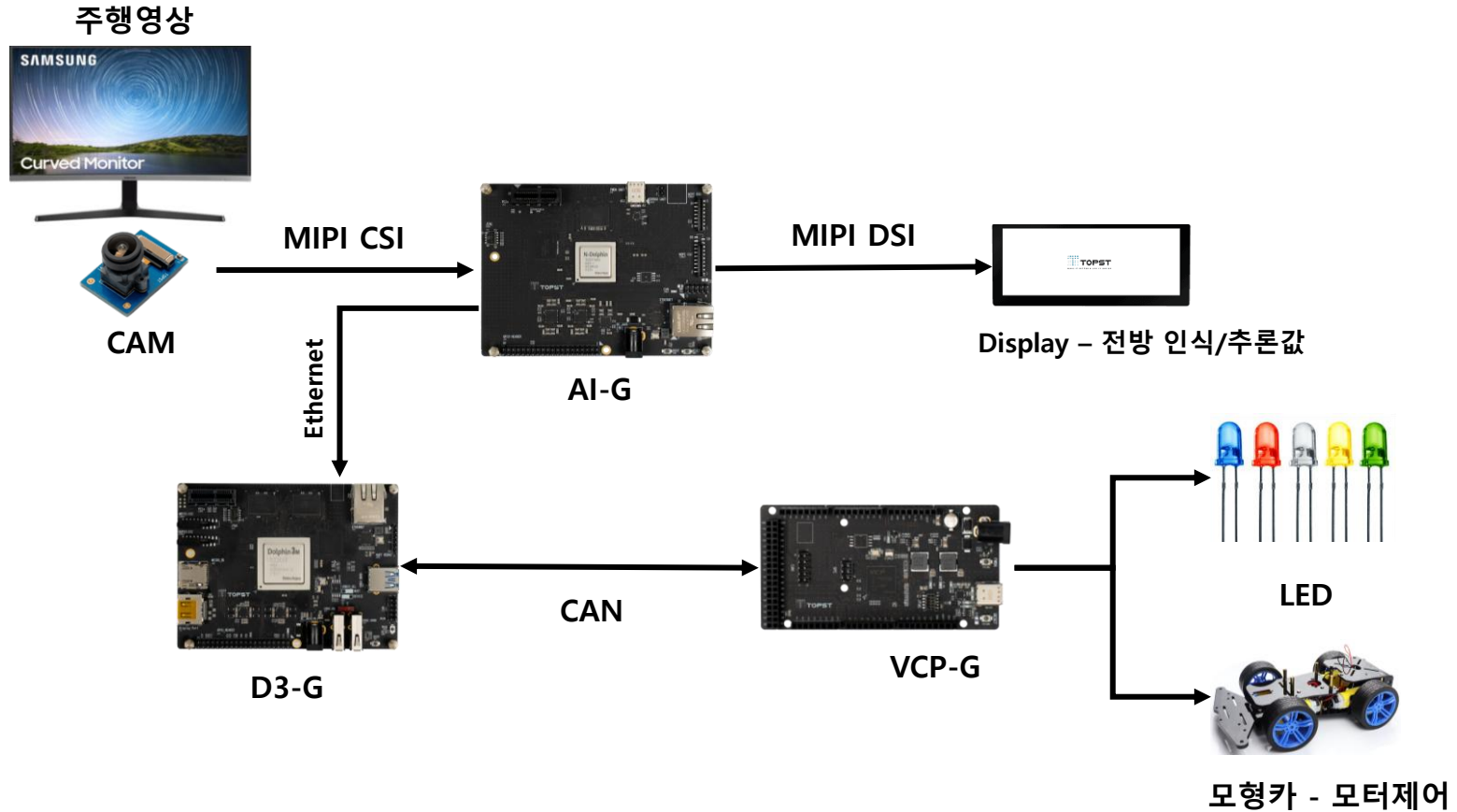
각 Zone controller가 body, chassis, power train 제어 기능을 통합

Central Computer - (I/O) Zonal Architecture

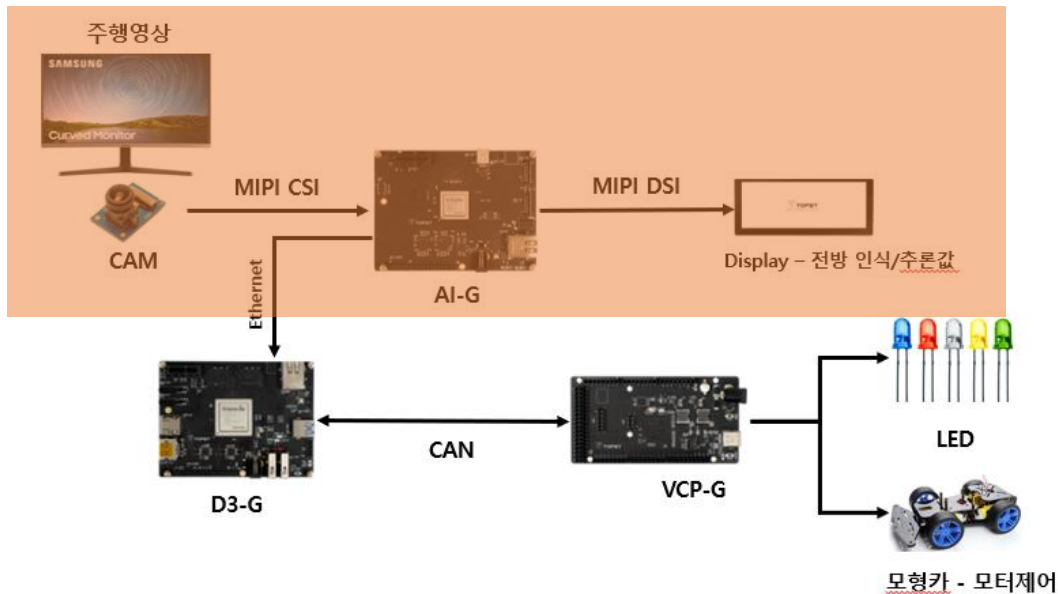


분산된 domain 제어 기능을 고성능 Central HPC로 통합,
zone controller는 I/O aggregation 기능

TOPST 제품 및 실습 과정 소개



TOPST 제품 및 실습 과정 소개



• 동작 시나리오(AI-G)

1. 영상 입력

- 전방 카메라를 통해 실시간 영상 데이터 받음

2. 객체 인식

- NPU보드에서 YOLOv8 모델을 통해 사람, 차량 등 객체 인식 추론 수행

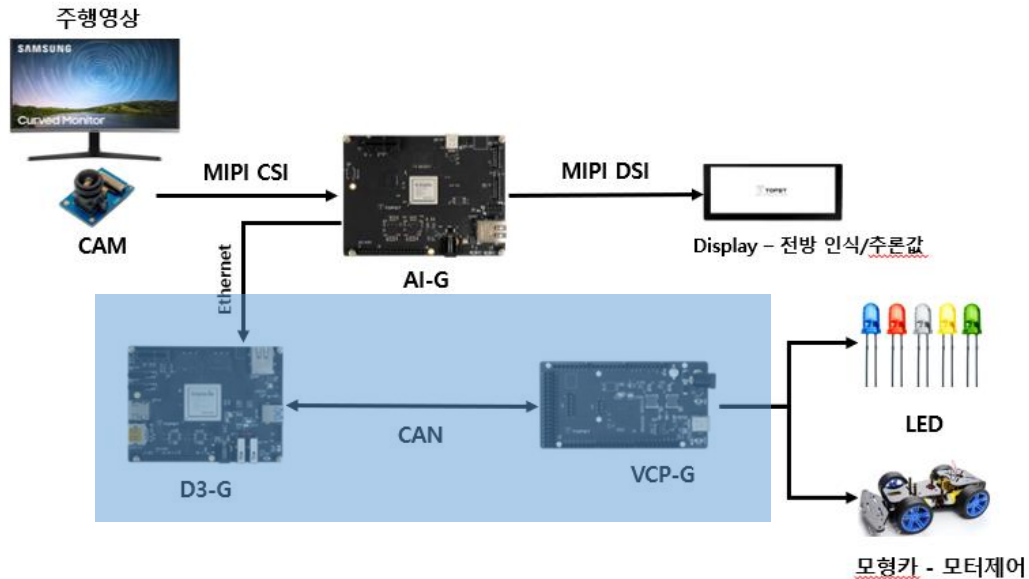
3. 추론 결과 시각화

- 객체 인식 결과(바운딩 박스, 클래스 정보)를 display를 통해 시각화

4. 추론 정보 전송

- 추론 결과 데이터를 이더넷을 통해 D3-G 보드로 전송

TOPST 제품 및 실습 과정 소개



• 동작 시나리오(D3-G)

1. 추론 결과 파싱

- AI-G로 부터 전달된 추론 결과를 수신 및 파싱

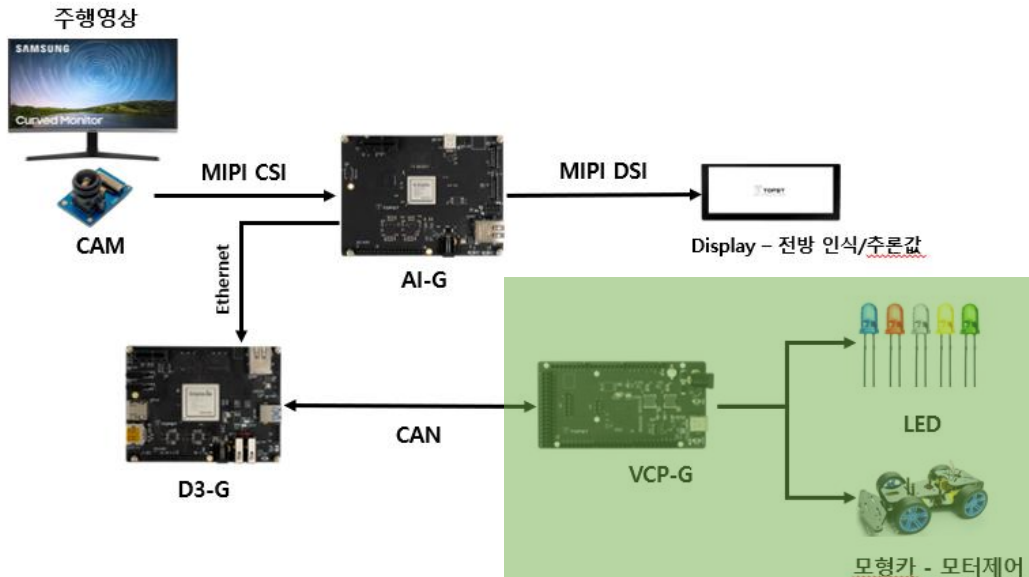
2. CAN 메시지 구성

- 파싱된 객체 정보를 기반으로 제어용 CAN 메시지를 생성

3. VCP-G로 메시지 전송

- 구성된 CAN 메시지를 CAN 통신을 통해 VCP-G로 전송

TOPST 제품 및 실습 과정 소개



• 동작 시나리오(VCP-G)

1. 초기 주행 상태
 - 조향각 0도 상태에서 DC 모터 정속 주행 수행
2. 장애물 인식
 - 전방 장애물 인식 시 D3-G에서 차량 제어 명령 송신(CAN)
3. 경고 표시
 - 장애물 인식 메시지 송신 후 LED 제어 (경고 표시)
4. 장애물 회피 동작
 - 장애물 인식 시 조향각 조절 및 DC 모터 감속 수행
5. 정상 주행 복귀
 - 일정 시간 이후 정상 주행

TOPST 제품 및 실습 과정 소개

일자	1교시	2교시	3교시	4교시	5교시	6교시
1일차	Introduction	RTOS 개발 환경	TOPST VCP-G 실습	GPIO/ADC 기반 센서 제어 실습	PWM/I2C 기반 센서 제어 실습	VCP-G 활용 추가 기능 구현
2일차	CNN 기반 NPU 소개 및 AI-G 구성	AI 모델 최적화 이론 및 툴킷 소개	AI-G 모델 변환 실습 (Yolov8)		카메라 기반 RTPM 추론 실습	D3-G 실습
3일차	CAN 기본 개요	CAN 통신 구조의 이해	VCP-G CAN 기반 SW 실습	AI 추론 결과 연동 및 구조 이해		사용자 정의 제어 시나리오 구현

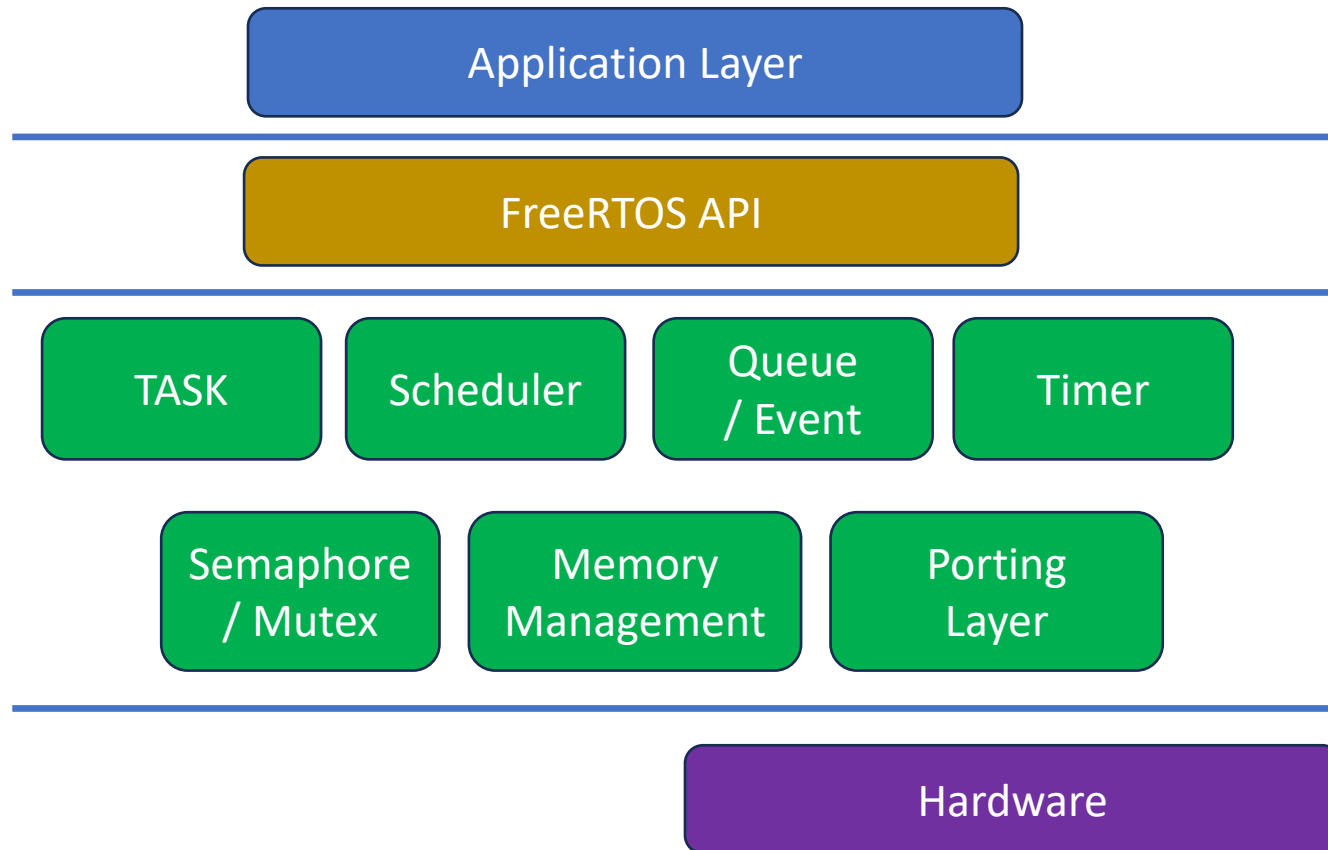
FreeRTOS



- **FreeRTOS(Free Real-Time Operate System)**

- 임베디드용 오픈소스 RTOS
- MCU(MicroController Unit) 환경에서 멀티태스킹, 스케줄링, 동기화, 시간제어 기능을 제공
 - 멀티태스킹 : 여러 작업(Task)을 동시에 실행하도록 관리
 - 스케줄링 : 각 작업의 **실행 순서와 우선순위**를 정함
 - 동기화 : 여러 작업이 **공유 자원**을 **충돌 없이** 사용
 - 시간제어 : 작업을 **지연, 주기적 실행, 타임아웃** 등으로 제어
- 커널 크기가 가볍고 빠름
- 사용 예
 - 산업 : 공장 자동화 정비, 로봇 제어기 등
 - 자동차 전장 : 조명/모터 컨트롤러, 와이퍼 등
 - IoT 디바이스 : 전등 스위치, 드론, 온도 센서 등

FreeRTOS 구조



FreeRTOS의 이해

- RTOS (Real-Time Operating System)
 - 정해진 시간 내에 반응해야 하는 시스템
 - 자동차 ECU, 모터 제어, 센서 수집, 통신 스택 등과 같은 실시간성 기능군
- 주요 기능

기능	설명
태스크(Task)	병렬로 동작하는 여러 실행 단위
스케줄러(Scheduler)	어떤 태스크가 CPU를 사용할지 결정
시간 관리(Timer)	주기적 타이머 인터럽트를 통해 태스크 전환
동기화(Semaphore, Mutex)	자원 공유 시 충돌 방지
통신(Queue, Event Group)	태스크 간 데이터 교환

FreeRTOS의 이해

- **Linux 와의 차이점**

- Linux에서는 bash, busybox 등 셸(shell)이 프로세스를 관리하고 실행 명령 전달

- FreeRTOS는 Shell 개념 부재

- **파일시스템 부재**

- 대부분 MCU는 NAND/NOR 플래시 기반

- OS가 관리하는 루트 파일시스템 없음

- 실행 가능한 "파일(executable)" 개념 없음

- **메모리 공간이 제한적**

- 수십 KB~수MB RAM 수준

- ELF 로더, 동적 링커 같은 기능을 둘 여유가 없음

- **MMU(메모리 보호)가 없음**

- 여러 프로세스를 독립적으로 격리할 수 없어 새로운 프로그램을 런타임에 "추가 실행" 허락하지 않음

- 즉, 정적으로 미리 빌드된 태스크를 실행하는 방식

FreeRTOS의 이해

- **FreeRTOS의 스케줄링 기본 개념**

- TASK에 Priority를 부여
- TASK Priority는 "누가 먼저 CPU를 쓸 수 있는가"를 결정
- FreeRTOS는 선점형(preemptive) 우선순위 기반 스케줄러를 사용
- 항상 가장 높은 우선순위 중 Ready 상태인(Task Ready) 태스크가 CPU를 점유
- 더 높은 우선순위의 태스크가 Ready 상태가 되면, 현재 실행 중인 낮은 우선순위 태스크를 즉시 중단(preempt) 시키고 실행
 - 단, 동일 순위의 TASK는 일정 시간씩 번갈아 실행

- **TASK의 실행 시간을 보장하지 않음**

- FreeRTOS는 Hard real-time 이 아닌 Soft real-time 스케줄러를 사용하는 OS

- **Hard real-time의 이해**

- TASK의 실행 시점과 지연(latency) 을 수학적으로 예측 가능하게 보장
- 시간이 초과될 경우, 시스템 실패로 간주
- Hard real-time OS의 예
 - VxWorks, QNX Neutrino, Integrity, SafeRTO 등

VCP-G Software Architecture

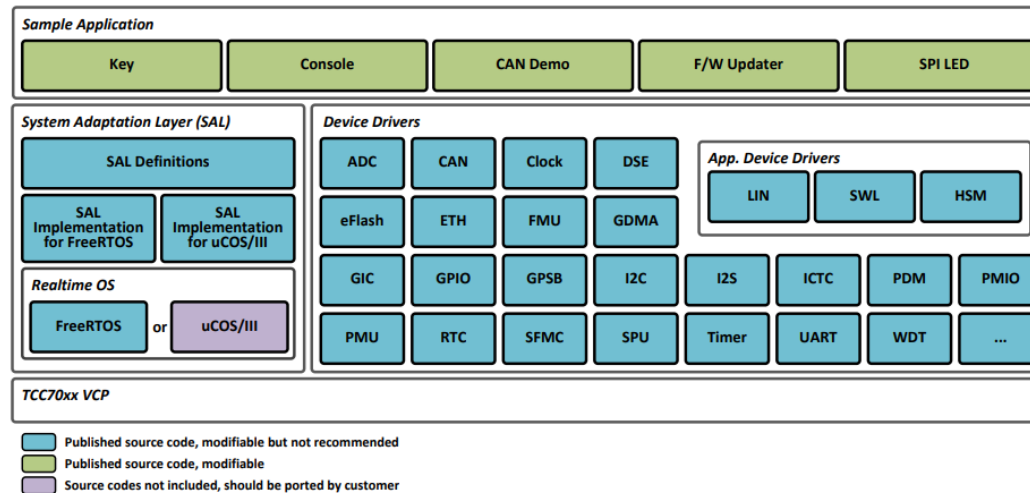
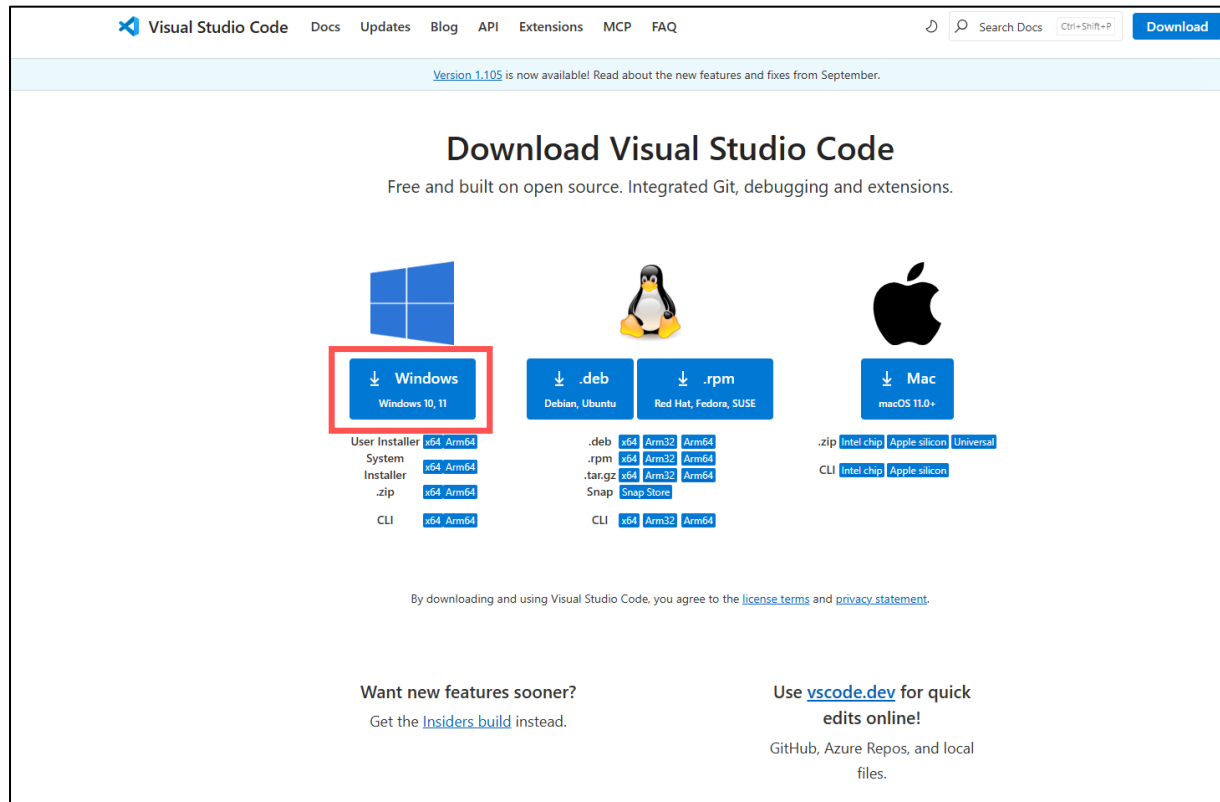


Figure 2.1 Block Diagram of Software Components in TCC70xx MCU BSP

- VCP-G에선 Realtime OS로 FreeRTOS & uCOS/III 지원
- 본 과정에선 FreeRTOS를 사용하여 GPIO 와 CAN 통신을 위한 예제만 사용

VSCode 에서 WSL 연결하기

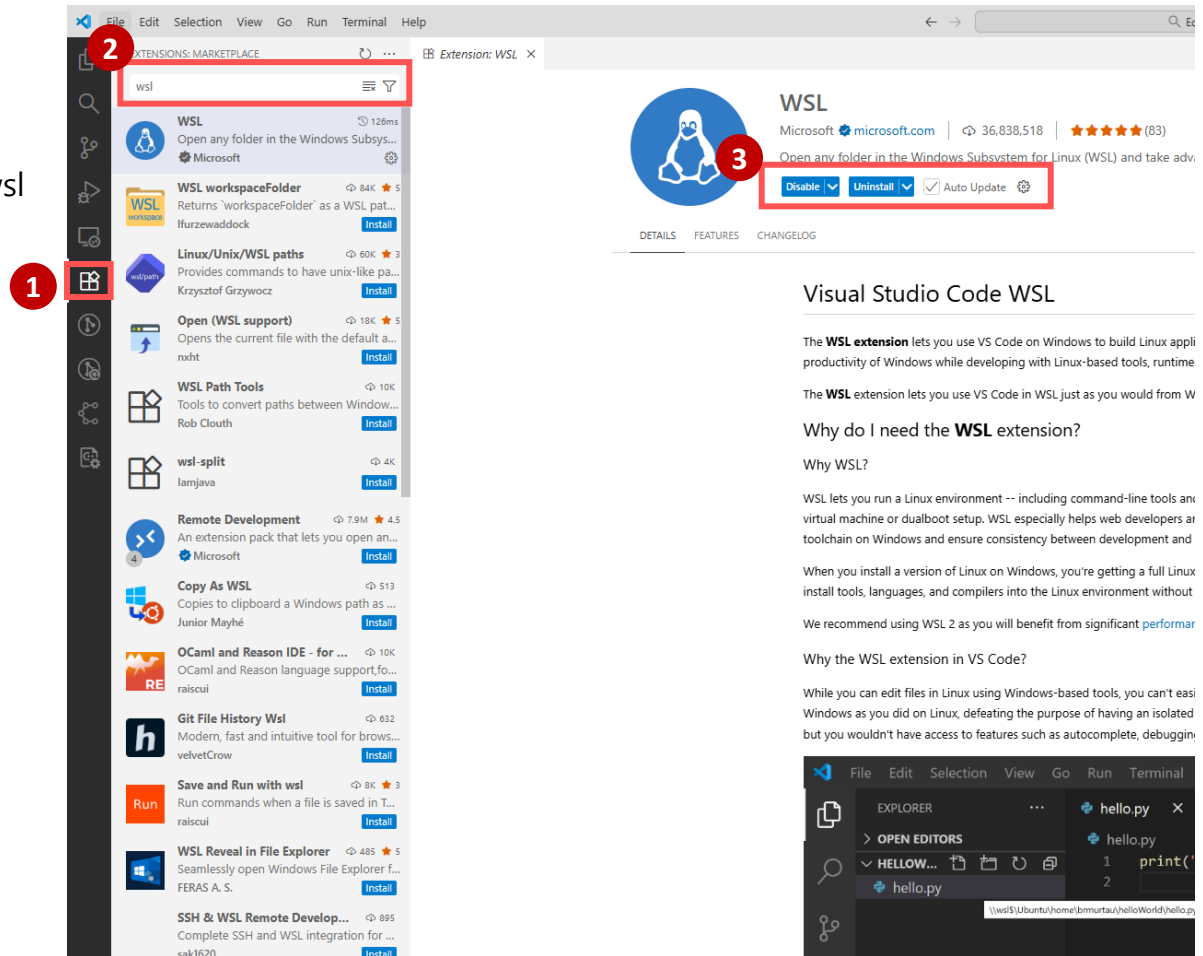
1. VSCode 설치
2. [Download Visual Studio Code - Mac, Linux, Windows](#)



VSCode 에서 WSL 연결하기

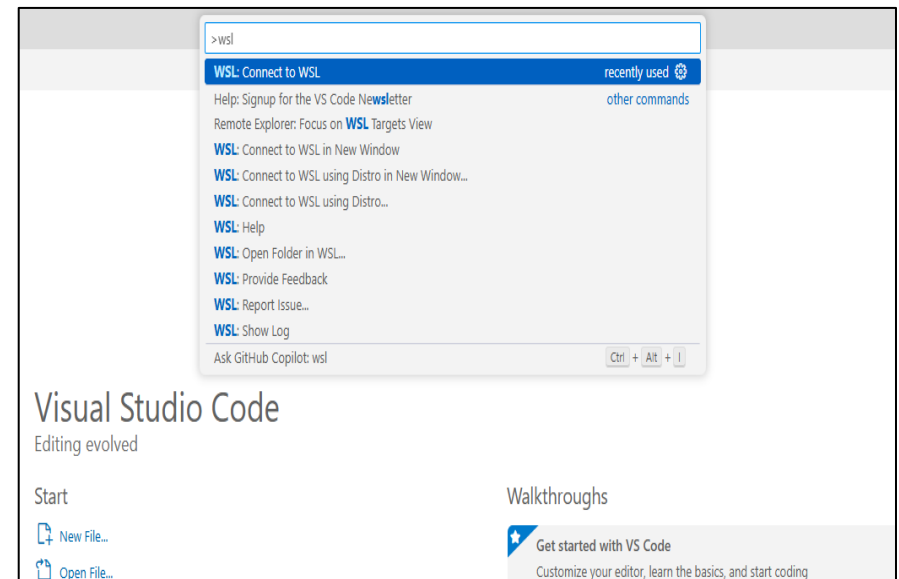
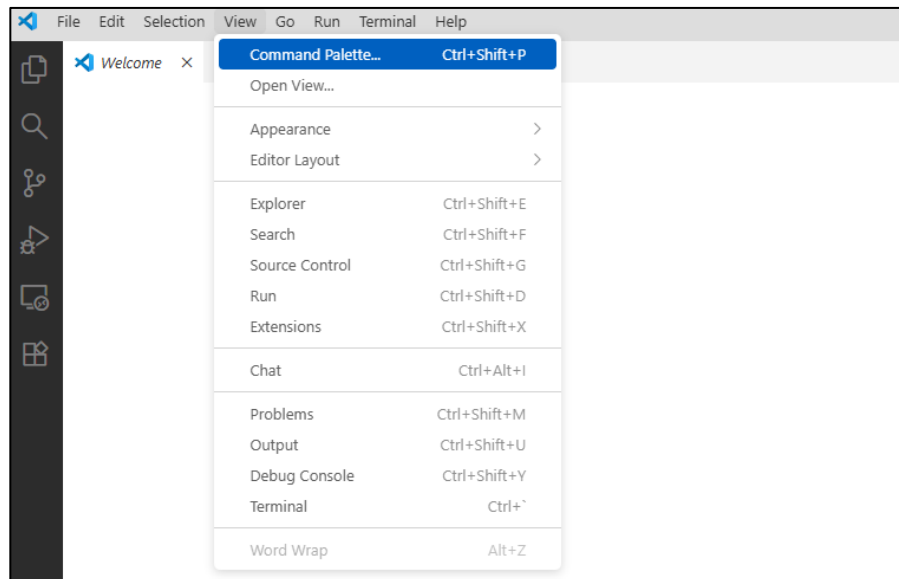
3. VSCode Extension에서 WSL 설치

1. 좌측 Extension Icon 클릭
2. Extension Marketplace에서 wsl
3. wsl install



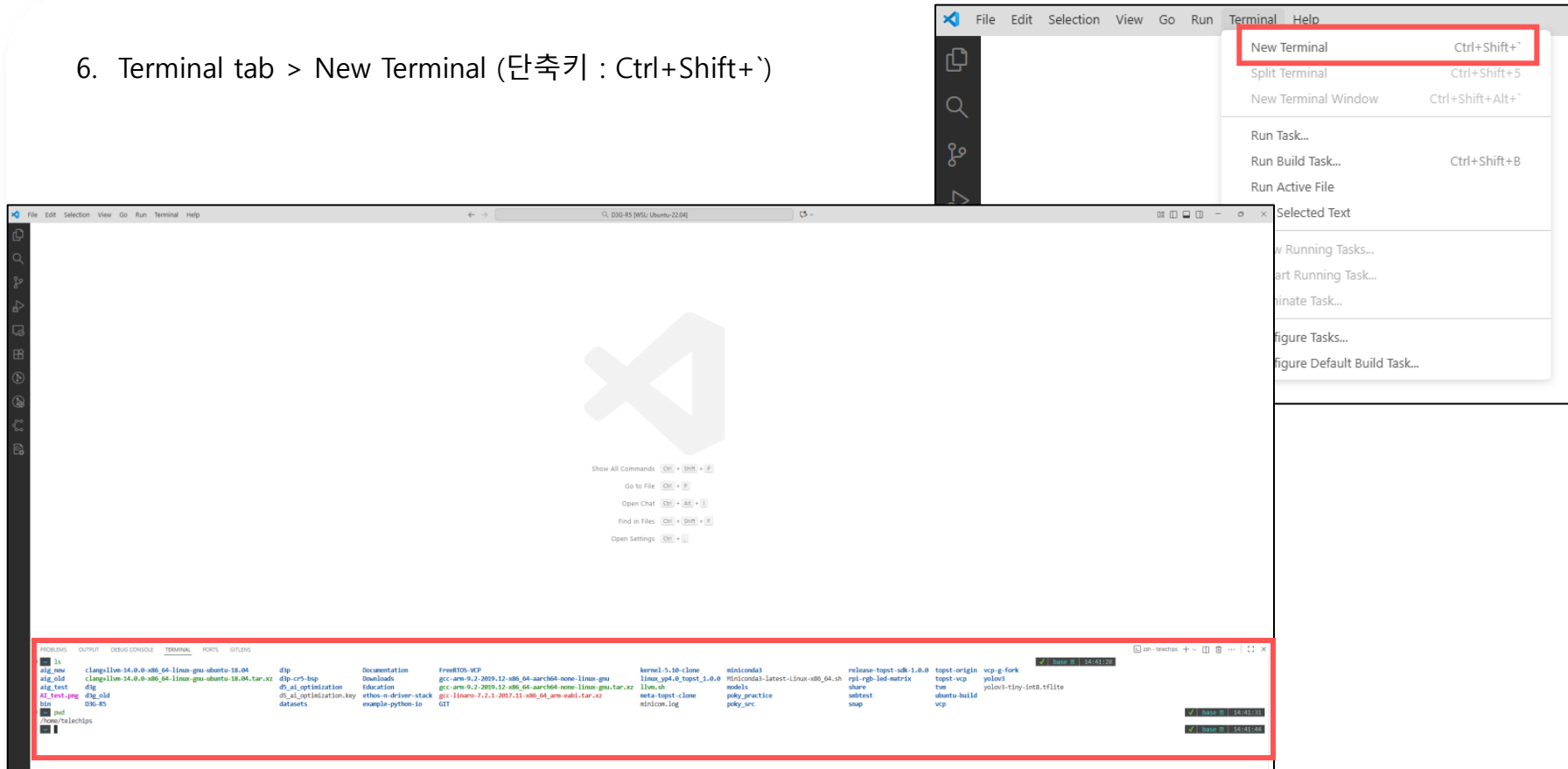
VSCode 에서 WSL 연결하기

4. 상단 View > command line palette 선택(단축키 : Ctrl + Shift + p)
5. 입력창에 wsl 입력 후, "WSL: Connect to WSL" 선택



VSCode 에서 WSL 연결하기

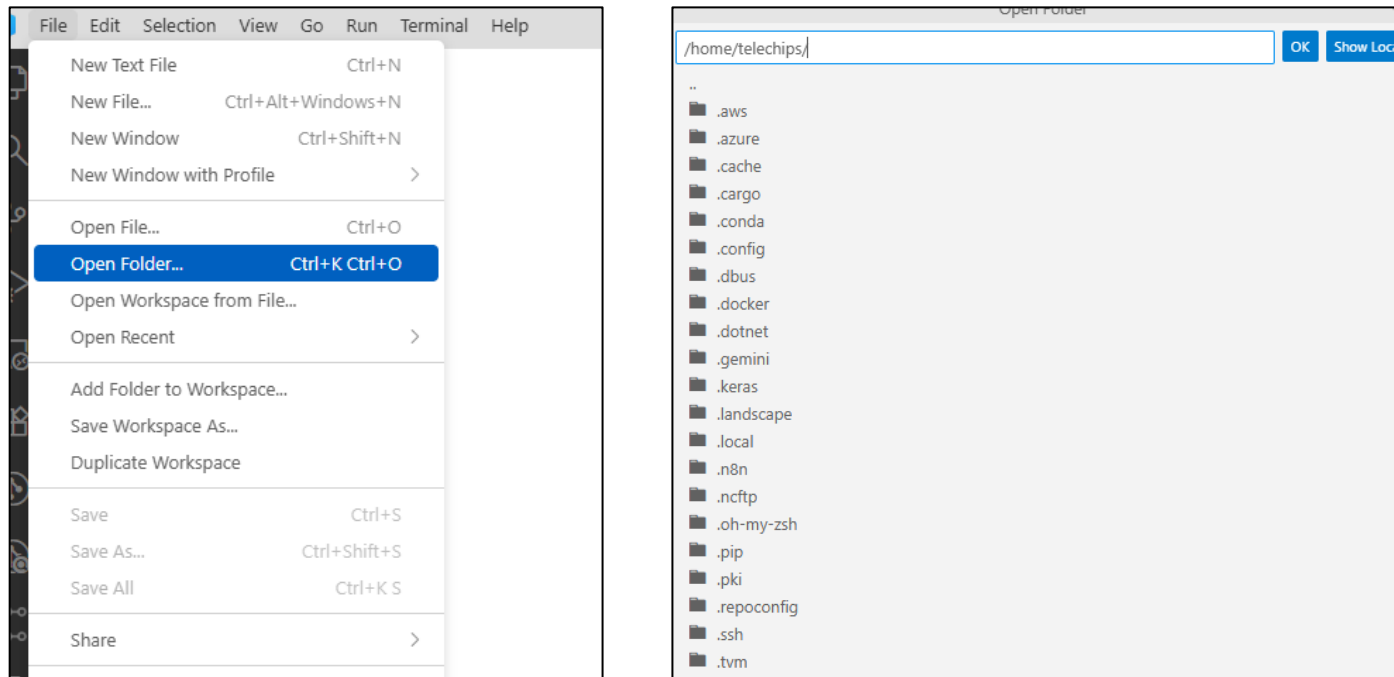
6. Terminal tab > New Terminal (단축키 : Ctrl+Shift+`)



원도우 명령 프롬프트에서 wsl 접속한 것과 동일한 환경 사용 가능

VSCode 에서 WSL 연결하기

7. File > Open Folder선택 하여, 원하는 폴더를 열 수 있음



폴더를 열어서 VScode 에서 파일 편집&저장 가능

FreeRTOS의 이해

• Toolchain Set Up

- VCP-G는 빌드 시 **gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi** Toolchain을 사용
- 하기 과정은 **WSL 환경에서 수행한다.**
- Download Toolchain
 - `$ cd`
 - `$ wget https://releases.linaro.org/components/toolchain/binaries/7.2-2017.11/arm-eabi/gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi.tar.xz`

```
ben@TC-E-20-0385 ~$ main wget https://releases.linaro.org/components/toolchain/binaries/7.2-2017.11/arm-eabi/gcc-
linaro-7.2.1-2017.11-x86_64_arm-eabi.tar.xz
--2025-11-06 11:17:41-- https://releases.linaro.org/components/toolchain/binaries/7.2-2017.11/arm-eabi/gcc-linaro-7.2.1-
2017.11-x86_64_arm-eabi.tar.xz
Resolving releases.linaro.org (releases.linaro.org)... 52.215.200.125
Connecting to releases.linaro.org (releases.linaro.org)|52.215.200.125|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://publishing-ie-linaro-org.s3.amazonaws.com/releases/components/toolchain/binaries/7.2-2017.11/arm-eabi/
gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi.tar.xz?Signature=HM80blw4NdWkERN66FKa5Kh04HQ%3D&Expires=1762395552&AWSAccessKey
Id=AKIAIELXV2RYNAHFUP7A [following]
--2025-11-06 11:17:42-- https://publishing-ie-linaro-org.s3.amazonaws.com/releases/components/toolchain/binaries/7.2-20
17.11/arm-eabi/gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi.tar.xz?Signature=HM80blw4NdWkERN66FKa5Kh04HQ%3D&Expires=17623955
52&AWSAccessKeyId=AKIAIELXV2RYNAHFUP7A
Resolving publishing-ie-linaro-org.s3.amazonaws.com (publishing-ie-linaro-org.s3.amazonaws.com)... 3.5.69.25, 3.5.68.65,
52.92.32.145, ...
Connecting to publishing-ie-linaro-org.s3.amazonaws.com (publishing-ie-linaro-org.s3.amazonaws.com)|3.5.69.25|:443... co
nnected.
HTTP request sent, awaiting response... 200 OK
Length: 274983024 (262M) [application/octet-stream]
Saving to: 'gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi.tar.xz'

7.11-x86_64_arm-eabi.tar.xz  25%[=====>] 67.93M 8.71MB/s eta 29s |
```

FreeRTOS의 이해

- **Toolchain Set Up**

- Extract the Toolchain (tar.xz파일 압축해제는 tar -xvf 명령어로 한다)
 - `$ tar -xvf gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi.tar.xz`
- Move the Toolchain to /opt (/bin에 있는 파일들 충돌 방지를 위해)
 - `$ sudo mv gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi /opt/`

FreeRTOS의 이해

```
ben@TC-E-20-0385 ~ > main > cd /opt/gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi
ben@TC-E-20-0385 /opt/gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi > ./bin/arm-eabi-gcc --version
arm-eabi-gcc (Linaro GCC 7.2-2017.11) 7.2.1 20171011
Copyright (C) 2017 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

- **Verify Toolchain**

- Toolchain이 정상적으로 설치되었는지 검증 필요
- Toolchain Directory로 이동
 - \$ cd /opt/gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi
- 설치된 GCC Compiler의 버전 확인
 - \$./bin/arm-eabi-gcc --version

VCP-G 환경 설정

```
ben@TC-E-20-0385 ~$ git clone git@github.com:topst-development/FreeRTOS-VCP.git topst-vcp
Cloning into 'topst-vcp'...
remote: Enumerating objects: 1221, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 1221 (delta 12), reused 4 (delta 0), pack-reused 1191 (from 1)
Receiving objects: 100% (1221/1221), 76.85 MiB | 9.18 MiB/s, done.
Resolving deltas: 100% (374/374), done.
ben@TC-E-20-0385 ~$ cd topst-vcp
ben@TC-E-20-0385 ~/topst-vcp$
```

- **Clone Source Code**

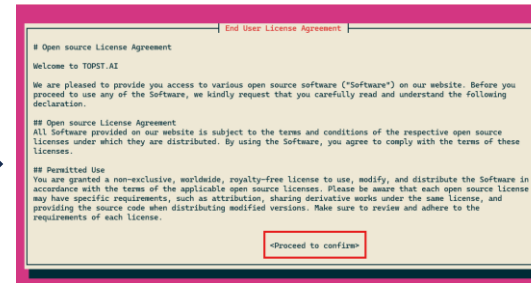
- WSL2에서 VCP-G FreeRTOS 소스 코드를 가져오기 위해 git clone
 - \$ cd
 - \$ git clone https://github.com/topst-development/FreeRTOS-VCP topst-vcp
- 소스 코드가 포함된 디렉터리로 이동 후 확인
 - \$ cd topst-vcp

VCP-G 환경 설정

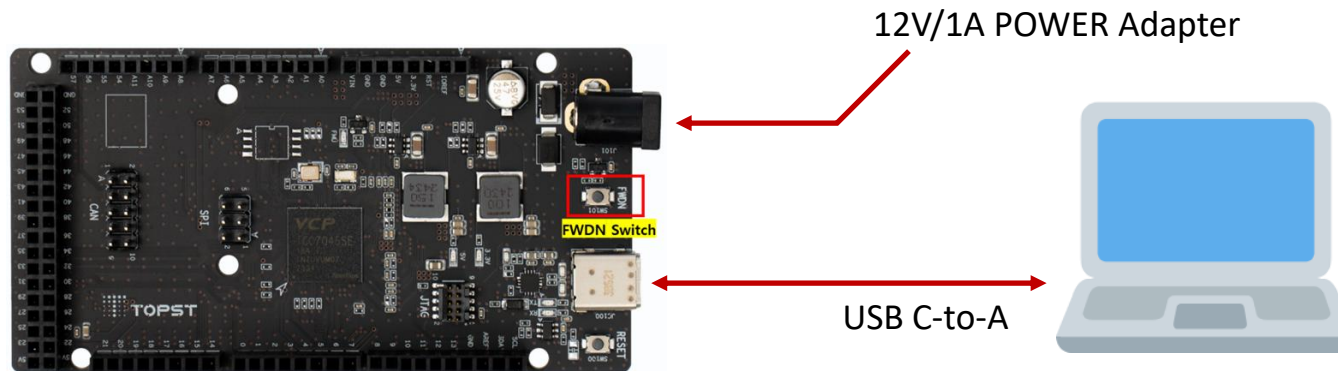
```
ben@TC-E-20-0385 ~/topst-vcp ♦ develop ./easy-setup_vcp-g.sh
cd build/tcc70xx/gcc/
make
ben@TC-E-20-0385 ~/topst-vcp ♦ develop cd ~/topst-vcp/build/tcc70xx/gcc
ben@TC-E-20-0385 ~/topst-vcp/build/tcc70xx/gcc ♦ develop make
```

• Build Guide

- Execute easy-setup_vcp-g.sh
 - \$./easy-setup_vcp-g.sh
- 라이선스 [Tab] -> [Enter]로 동의 후 build 진행
 - \$ cd build/tcc70xx/gcc
 - \$ make
- Build 후 생성된 output 디렉터리에서 ROM 파일 확인 가능
 - \$ ls -al output/



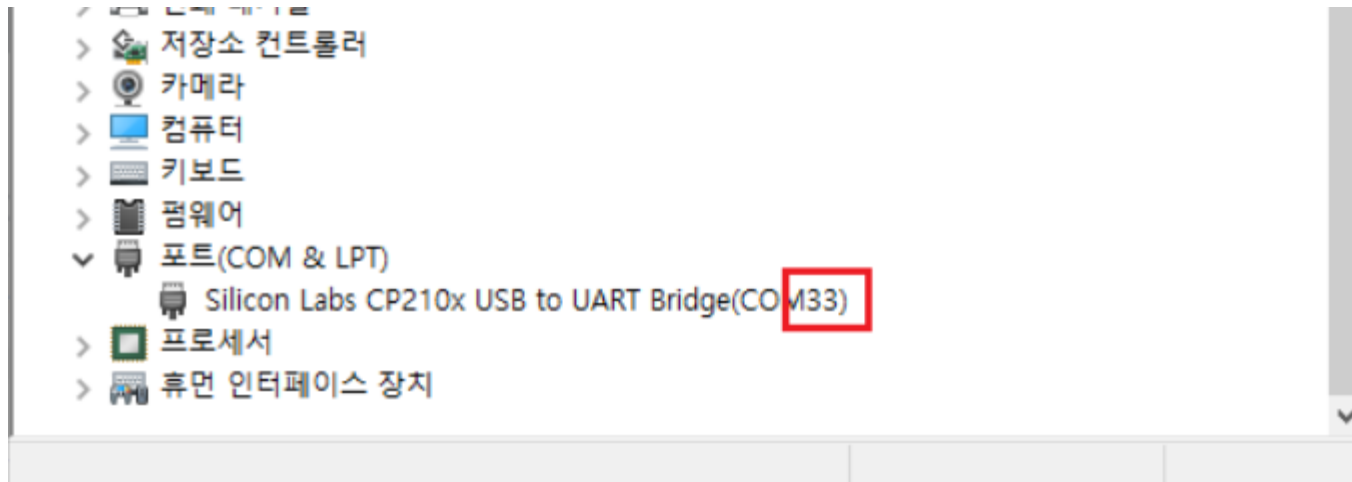
VCP-G 환경 설정



- **FWDN (FreeRTOS)**

- USB Type-C 케이블을 Host PC 및 VCP-G의 USB Type-C FWDN 포트에 연결
- FWDN 스위치를 누른 상태에서 전원 케이블을 VCP-G 보드에 연결 (**FWDN 모드 진입**)
 - VCP-G 의 전원 케이블은 **12V/1A 케이블만 사용**

VCP-G 환경 설정



• 장치 드라이버 설치

- <https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads>
- [CP210x Windows Drivers](#) 다운로드
- 압축해제 후 CP120xVCPInstaller_x64 설치

VCP-G 환경 설정

선택 관리자: Windows PowerShell

```
PS C:\Windows\system32> usbipd list
Connected:
BUSID VID:PID DEVICE STATE
3-1 0bda:8153 Realtek USB GbE Family Controller Not shared
5-3 04e8:a057 Samsung USB C Earphone, USB 입력 장치 Not shared
5-7 27c6:6594 Goodix MOC Fingerprint Not shared
5-9 5986:1199 Integrated Camera, Integrated IR Camera, Camera DFU Device Not shared
5-10 8087:0036 인텔(R) 무선 Bluetooth(R) Not shared
5-13 0bda:8153 Realtek USB GbE Family Controller #2 Not shared
6-2 17ef:30b0 ThinkPad USB-C Dock Audio, USB 입력 장치 Not shared
6-3 17ef:30a9 Billboard Device, Vendor Interface Not shared
7-2 10c4:ea60 Silicon Labs CP210x USB to UART Bridge(COM7) Shared
7-3 067b:2303 Prolific USB-to-Serial Comm Port(COM18) Not shared
7-4 067b:2303 Prolific USB-to-Serial Comm Port(COM17) Not shared
9-2 046d:c52b Logitech USB Input Device, USB 입력 장치 Not shared
9-3 046d:c548 Logitech USB Input Device, USB 입력 장치 Not shared
```

```
ben@TC-E-20-0385 ~/$ sudo dmesg | grep tty
[ 88.186711] usb 1-1: cp210x converter now attached to ttyUSB0
ben@TC-E-20-0385 ~/$
```

• Connect Hardware to Host PC

- Run PowerShell and attach the VCP-G (**Windows환경에서** PowerShell 실행-관리자권한)
 - \$ winget install --interactive --exact dorssel.usbipd-win -> powershell 재시작
 - \$ usbipd list
 - \$ usbipd bind --busid <busid>
 - \$ usbipd attach --wsl --busid <busid>
- Connect USB Type-C Cable & Verify USB Connection (**WSL2[Ubuntu]에서 실행**)
 - \$ sudo apt-get install usbutils && lsusb
 - \$ sudo dmesg | grep tty

VCP-G 환경 설정

```
ben@TC-E-20-0385 ~/topst-vcp/build/tcc70xx/gcc ♦ develop ± sudo ~/topst-vcp/tools/fwdn_vcp/fwdn --fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom -w output/tcc70xx_pflash_boot_2M_ECC.rom
[main:27] FWDN VCP v0.2.0 - 2022.8.12 11:59:48
[OpenPort:23] Complete open port(/dev/ttyUSB0)
[StartVCPFWDN:45] Complete to receive start res
[LoadFwdnFW:144] Complete to send start msg
[CheckResPacket:172] Complete to receive ack for cmd(0xFFFF0000(RECEIVE_HSM_CMD))
[WriteFile:284] Complete to send command(0xFFFF0000(RECEIVE_HSM_CMD))
11% [||||] ] 15192/131072
```

•
•
•

```
[WriteFile:295] Complete to send file - output/tcc70xx_pflash_boot_2M_ECC.rom
[main:137] Complete FWDN
[PrintCurTime:99] 25/11/06-11:30:03
```

• FWDN (VCP-G FreeRTOS)

- Execute the Download Command (WSL2)
 - \$ sudo ~/topst-vcp/tools/fwdn_vcp/fwdn
 - fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom
 - w ~/topst-vcp/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom
- FWDN이 끝난 후 Powe cable을 재연결 or RESET 버튼 토글

VCP-G 환경 설정

```
D|E|R=00011f00
S0V=56010000
RM=00000000
LOAD
MT
MR0=0000a042
MR1=00020018
MR2=00000000
HSM_V_OK
```

```
Initialize System done
Welcome to Telechips MCU BSP
```

```
=====
MCU BSP Version: V1.0.0
-----
CHIP   NAME   : 17045
DUAL   BANK   : 0
EXPAND FLASH : 1
REMAP  MODE   : 0
HSM    READY  : 1
=====
```

```
(D)[SAL ][FR_OSSstart:504] ~ Done to initialize Free RT Operating System ~
```

```
[PMIO][PMIO_SetNotiGpkIrq:1118] Set notification of gpk irq for application.
[PMIO][PMIO_SetNotiGpkIrq:1118] Set notification of gpk irq for application.
```

```
> help
```

```
=====
===== How to use command =====
=====
[command_string] [arg1] ... [arg10]

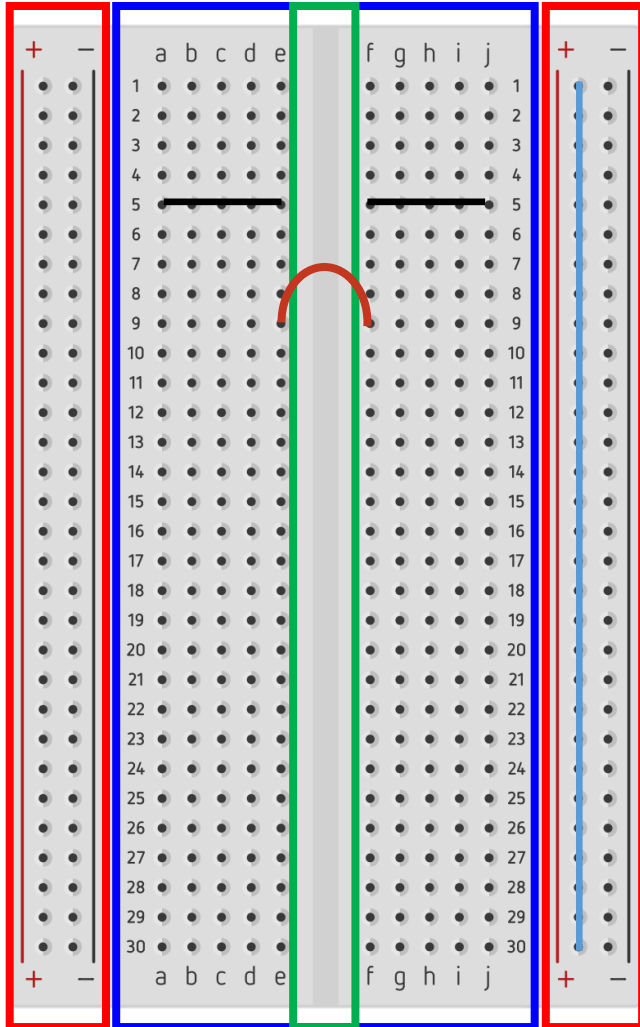
1. display alive message : [alive] [on/off]

> █
```

• Verify Software on Board (WSL2)

- Install minicom
 - \$ sudo apt install minicom
- Open a Serial Connection
 - \$ sudo minicom -D /dev/ttyUSB0 -b 115200 -8

Bread Board 이해

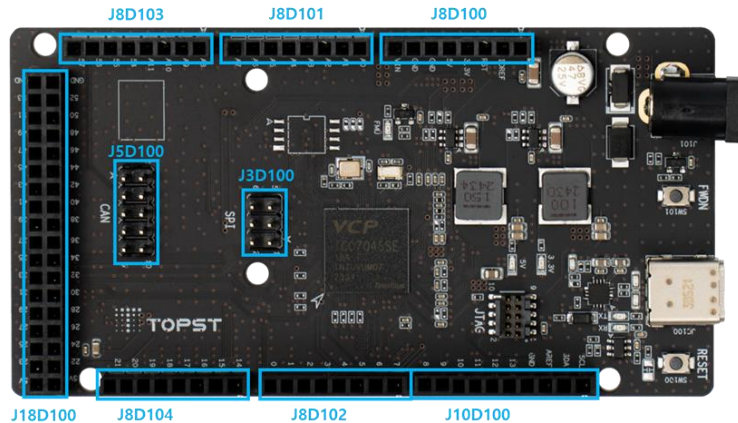


- 기판에 납땜을 하지 않고도 회로를 구성할 수 있는 회로 구성용 도구
- 파란색 공간은 초록색 공간인 홈을 기준으로 좌우가 줄마다 가로로 선이 이어져 있음
 - 좌 우를 연결하려면 e와 f라인을 연결하여 사용하여야 함
- 빨간색 공간은 +, - 각각 세로로 선이 이어져 있음
 - 전원 공급을 위해 사용함

GPIO 이해

- **GPIO(General-Purpose Input Output)**

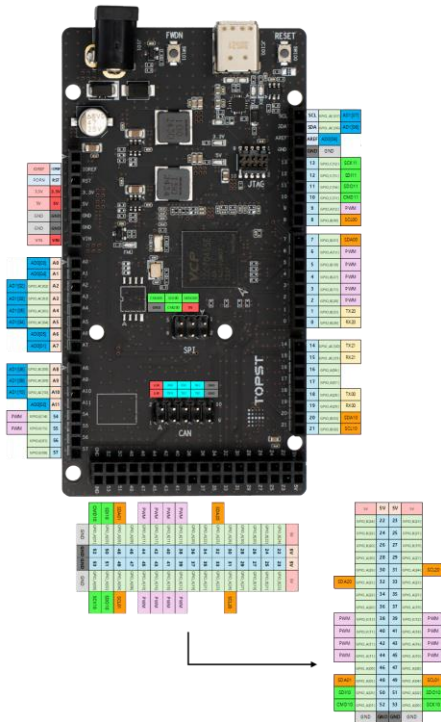
- 범용 입력 출력
- 마이크로 프로세서가 주변장치와 통신하기 위해 범용으로 사용되는 입/출력 포트



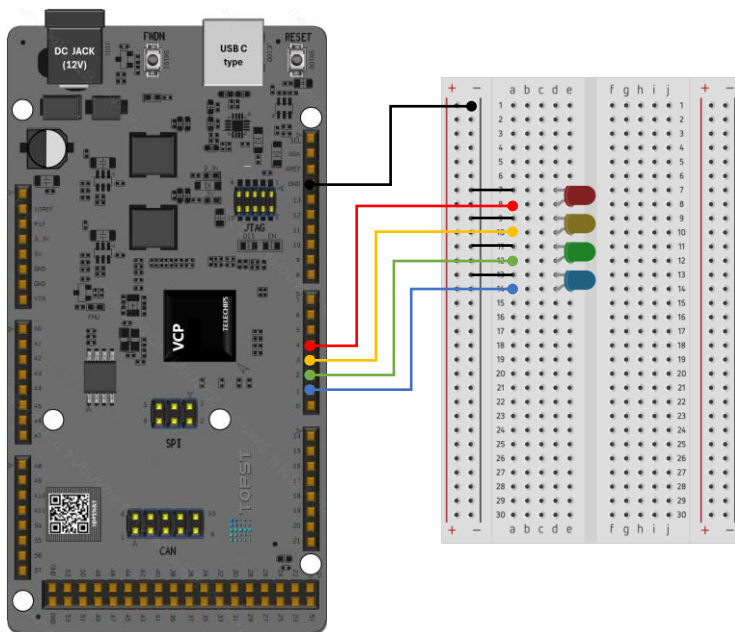
- 포트에 대해 입력, 출력을 설정할 수 있음
- 사용 전, GPIO 포트에 대한 설정을 반드시 지정해야 함
- 3.3V I/O interface(5V 입력 주의)
- 센서 : Gas, LED, Pressure ...

VCP-G 핀맵

- TOPST DOCS에서 VCP-G 핀 FUNC 및 GPIO 번호 확인
 - <https://topst.ai/tech/docs?page=1714a3dcf19dbf61088d7364d61e29f90a15ff2>



회로 구성 (VCP-G ↔ Bread Board, LED)



Pin Name	VCP-G Board	GPIO
LED_Red(+)	4	B[27]
LED_Yello(+)	3	B[11]
LED_Green(+)	2	B[28]
LED_Blue(+)	1	B[25]

• Experiment circuit

- Requirement
 - Breadboard
 - LED * 4
- GPIO Out

VIM 사용법



- **VIM 에디터 사용법**

- VIM 설치
 - `$ sudo apt-get install vim`
- VIM으로 파일 열기
 - `$ vim ~/test.c`
- 입력 : "i"
- 입력 취소 : "esc 키"
- 저장 후 종료 : ":wq"
- 지우기 : i(insert) 활성화 된 상태에서 "Backspace 키 혹은 Delete 키"

GPIO Code – led.h / led.c

~/topst-vcp/sources/app.sample/test.app gpio/led.h

```
#ifndef MCU_BSP_GPIO_LED_H
#define MCU_BSP_GPIO_LED_H

void LED_Init(void);

void LED_Run(void); // main task에서 호출할 진입 함수

#endif
```

~/topst-vcp/sources/app.sample/test.app gpio/led.c

```
#include "led.h"
#include <gpio.h>
#include <stdint.h>

static const uint32_t led_pins[4] = {
    GPIO_GPB(25),
    GPIO_GPB(28),
    GPIO_GPB(11),
    GPIO_GPB(27), // 4개 디지털 핀 GPIO 설정
};

void LED_Init(void)
{
    for (int i = 0; i < 4; ++i) {
        GPIO_Config(led_pins[i], (GPIO_FUNC(0) | GPIO_OUTPUT));
        GPIO_Set(led_pins[i], 0); } // GPIO 출력 설정, 초기 OFF
    }

void LED_Run(void)
{
    LED_Init(); // 초기화
    while (1) {
        for (int i = 0; i < 4; i++) {
            GPIO_Set(led_pins[i], 1); // 순서대로 ON
            SAL_TaskSleep(500); } // 500ms 대기
        for (int i = 3; i >= 0; i--) {
            GPIO_Set(led_pins[i], 0); // 순서대로 OFF
            SAL_TaskSleep(500); } // 500ms 대기
    }
}
```

GPIO Code – rules.mk / main.c

~/topst-vcp/sources/app.sample/test.app/gpio/rules.mk

```
SRCS += gpio_test.c
//추가
SRCS += led.c
```

~/topst-vcp/sources/app.sample/app.base/main.c

```
#include "led.h" //추가

static void Main_StartTask(void * pArg)
{
    (void)pArg;
    (void)SAL_OsInitFuncs();

    //삭제
    /* Service Init*/

    /* Create application tasks*/
    AppTaskCreate();

    while(1)
    { /*Task body, always written as an infinite loop.*/
        DisplayAliveLog();
        //mcu_printf("\n MCU Idle !!!«);
        (void)SAL_TaskSleep(5000);
    }
    //추가
    LED_Run();
}
```

Build

- build 진행
 - `$ cd ~/topst-vcp/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/topst-vcp/tools/fwdn_vcp/fwdn`
`--fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/topst-vcp/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- 보드 재부팅 시 LED가 순차 점등 및 소등 반복.

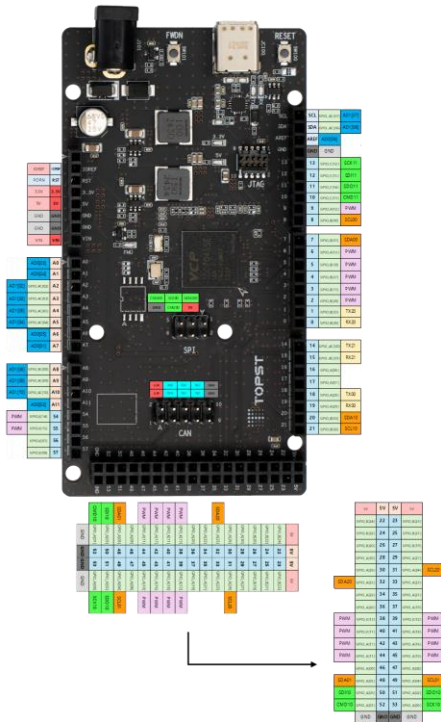
ADC 이해

- **ADC(Analog to Digital Converter)**
 - 연속적인 아날로그 신호를 이산적인 디지털 값으로 변환
 - 센서데이터를 기계가 이해할 수 있도록 변환 필요
 - 변환 과정
 - 1) 샘플링 – 일정 시간 간격으로 아날로그 값을 채취
 - 2) 양자화 – 측정 값을 여러 구간 중 하나로 근사
 - 3) 부호화 – 이 구간을 이진수로 표현
 - 출력 단위 : N비트 분해능일 경우, $2^N - 1$

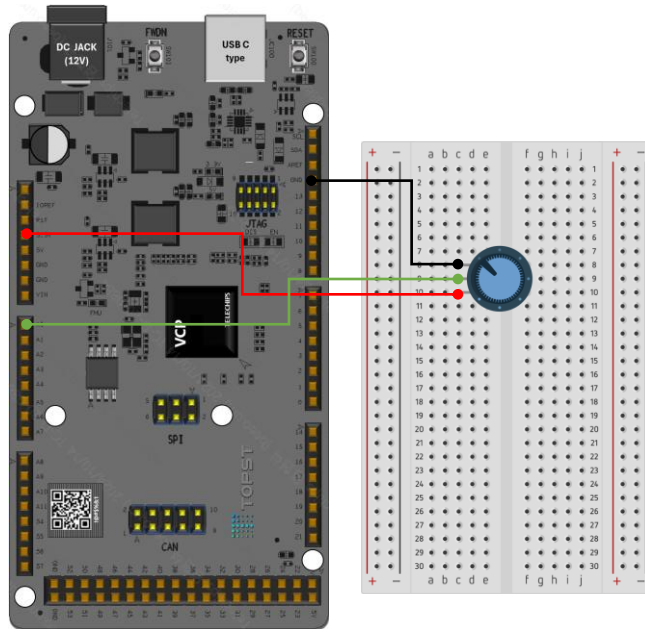


VCP-G 핀맵

- TOPST DOCS에서 VCP-G 핀 FUNC 및 GPIO 번호 확인
 - <https://topst.ai/tech/docs?page=1714a3dcdf19dbf61088d7364d61e29f90a15ff2>



회로 구성 (VCP-G ↔ Bread Board, Potentiometer)



- Experiment circuit
 - Requirement
 - Breadboard
 - Potentiometer * 1
 - ADC 사용

Pin Name	VCP-G Board	GPIO
Potentiometer left	GND	-
Potentiometer Middle	A0	AD0[03]
Potentiometer right	3.3V	-

VIM 사용법



- **VIM 에디터 사용법**

- VIM으로 파일 열기
 - `$ vim ~/test.c`
- 입력 : "i"
- 입력 취소 : "esc 키"
- 저장 후 종료 : ":wq"
- 지우기 : i(insert) 활성화 된 상태에서 "Backspace 키 혹은 Delete 키"

ADC Code

~/topst-vcp/sources/dev.drivers/adc/adc.h에 아래와 같이 명시

```
#define ADC_MODULE_0          (0U)
#define ADC_MODULE_1          (1U)

typedef enum ADCChannel
{
    ADC_CHANNEL_0              = 0,
    ADC_CHANNEL_1              = 1,
    ADC_CHANNEL_2              = 2,
    ADC_CHANNEL_3              = 3,
    ADC_CHANNEL_4              = 4,
    ADC_CHANNEL_5              = 5,
    ADC_CHANNEL_6              = 6,
    ADC_CHANNEL_7              = 7,
    ADC_CHANNEL_8              = 8,
    ADC_CHANNEL_9              = 9,
    ADC_CHANNEL_10             = 10,
    ADC_CHANNEL_11             = 11,
    ADC_CHANNEL_12             = 12,
    ADC_CHANNEL_13             = 13,
    ADC_CHANNEL_MAX            = 14
} ADCChannel_t;
```

AD03핀을 사용할 것이기 때문에, Module은 0, Channel은 3을 사용
(ex, AD14핀 사용은 Module 1, Channel 4 사용)

ADC Code – pot.h / pot.c

~/topst-vcp/sources/app.sample/test.app.adc/pot.h

```
#ifndef MCU_BSP_POT_H
#define MCU_BSP_POT_H

void Potentiometer_Run(void); // main task에서 호출할 진입
함수

#endif
```

~/topst-vcp/sources/app.sample/test.app.adc/pot.c

```
#include <adc.h>
#include <stdint.h>
#include "pot.h"

#define ADC_TYPE          0
#define ADC_MODULE        0 // 사용할 ADC Module 번호
#define ADC_CHANNEL        ADC_CHANNEL_3 // 측정할 채널

void Potentiometer_Run(void)
{
    uint32_t adc_val = 0; // ADC 변환 결과(12bit data)를
저장할 변수

    ADC_Init(ADC_TYPE, ADC_MODULE); // ADC 초기화

    while (1)
    {
        adc_val = ADC_Read(ADC_CHANNEL, ADC_MODULE, 0);
        // ADC 값 읽기
        adc_val &= 0xFFF; // 상위 상태 비트 제거 -> 하위
12bit(실제 ADC 값)만 추출

        mcu_printf("[ADC] CH%d = %4d\n", ADC_CHANNEL,
adc_val); // 결과를 UART 콘솔로 출력

        SAL_TaskSleep(100);
    }
}
```

ADC Code – rules.mk / main.c

~/topst-vcp/sources/app.sample/test.app.adc/rules.mk ~/topst-vcp/sources/app.sample/app.base/main.c

```
SRCS += adc_test.c
```

```
//추가
```

```
SRCS += pot.c
```

```
#include "pot.h" // 변경
```

```
Static void Main_StartTask(void * pArg)
```

```
{
```

```
    (void)pArg;
```

```
    (void)SAL_OsInitFuncs();
```

```
    // 변경
```

```
    Potentiometer_Run();
```

```
}
```

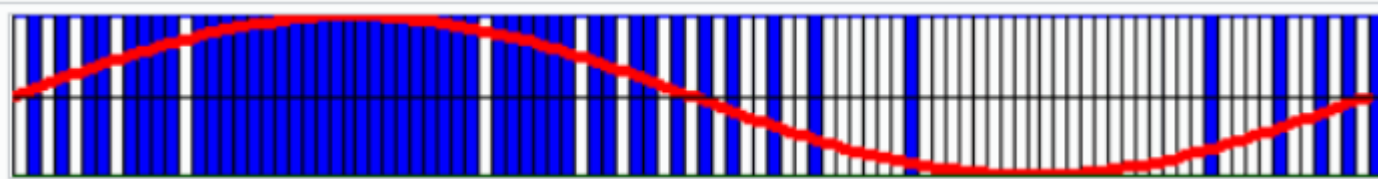
Build

- build 진행
 - `$ cd ~/topst-vcp/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/topst-vcp/tools/fwdn_vcp/fwdn`
`--fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/topst-vcp/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- `sudo minicom -D /dev/ttyUSB0 -b 115200 -8` 실행 후 보드 재부팅 시 UART 로그에 가변저항 값 출력.

PDM 이해

- **PDM(Pulse Density Modulation)**

- Pulse : 사각파, Density : 밀도, Modulation : 변조
사각파의 밀도를 제어한다는 의미

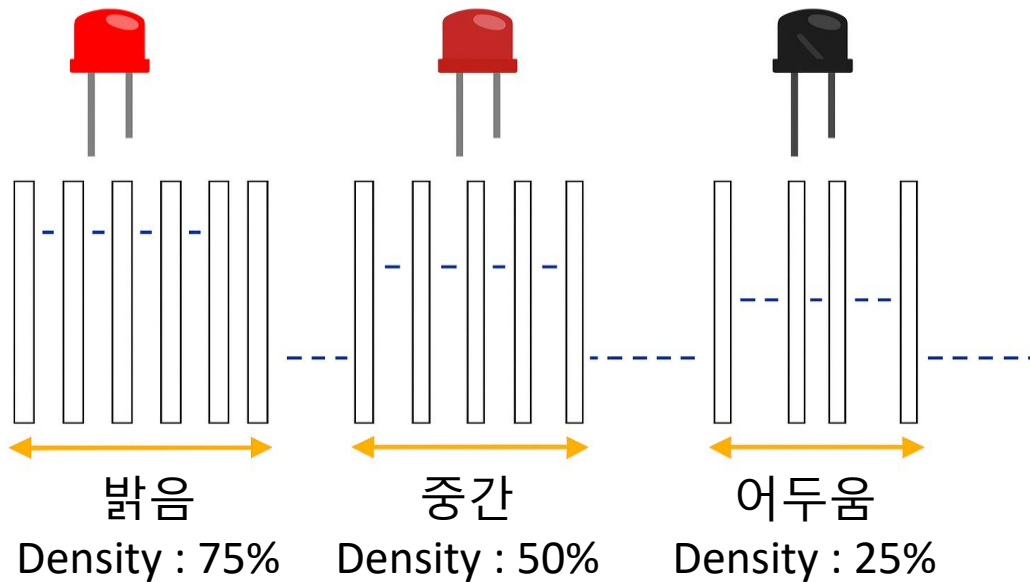


- 2개의 파라미터
 - 주파수 [Hz] : 1초에 생성되는 비트 스트림의 샘플링 속도
 - 밀도 [%] : 비트 스트림 내에서 '1'의 밀도
- 디지털 출력만으로 아날로그 신호를 낼 수 있는 기법

PDM 이해

- **PDM(Pulse Density Modulation)**

- HIGH와 LOW를 빠르게 반복함. 이 때 HIGH의 밀도를 조절



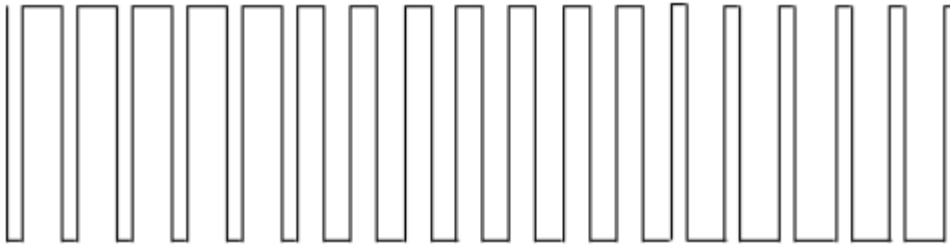
- 센서 : 마이크, 오디오 변환 ...

PWM 이해

- **PWM(Pulse Width Modulation)**

- Pulse : 사각파, Width : 폭, Modulation : 변조

사각파의 폭을 제어한다는 의미

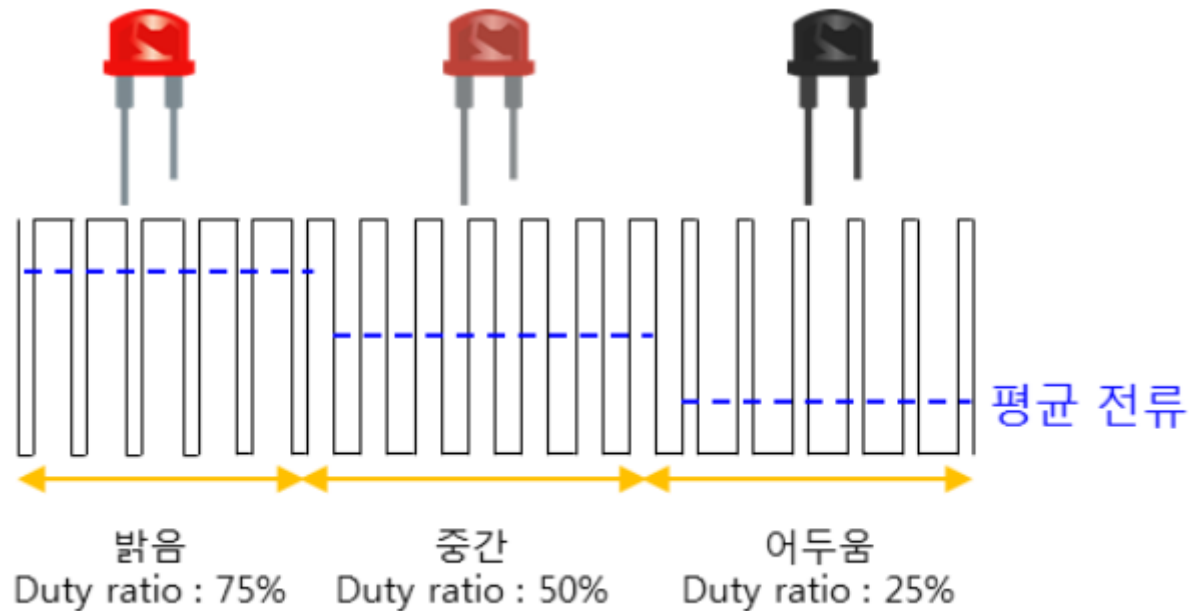


- 2개의 파라미터
 - 주파수 [Hz] : 1초에 진동하는 횟수
 - Duty ratio [%] : 한 주기 안에서 신호가 on 되어 있는 비율
- 디지털 출력만으로 아날로그 신호를 낼 수 있는 기법

PWM 이해

- PWM(Pulse Width Modulation)

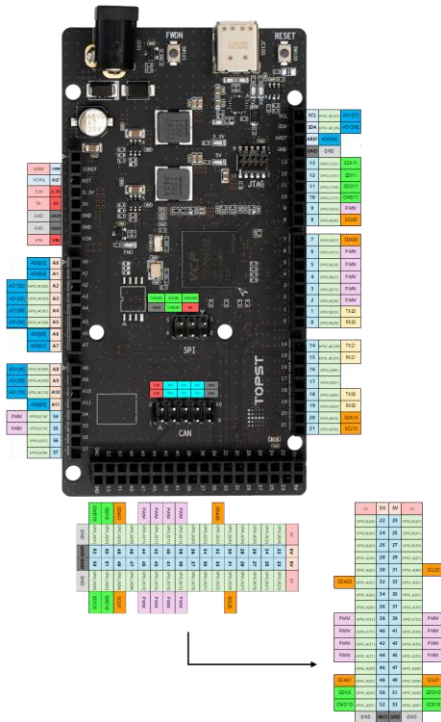
- HIGH와 LOW를 빠르게 반복함. 이 때 HIGH가 되는 시간을 조절



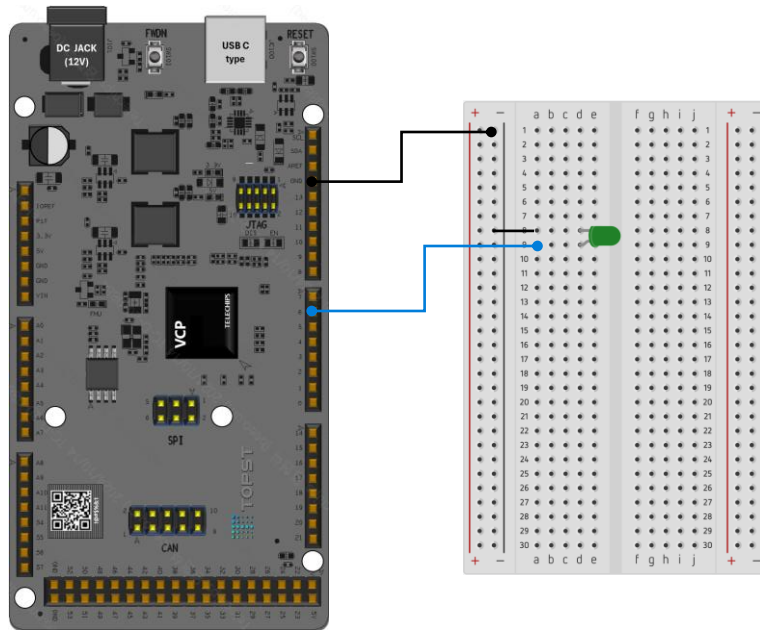
- 센서 : Led, Motor ...

VCP-G 핀맵

- TOPST DOCS에서 VCP-G 핀 FUNC 및 GPIO 번호 확인
 - <https://topst.ai/tech/docs?page=1714a3dcdf19dbf61088d7364d61e29f90a15ff2>



회로 구성 (VCP-G ↔ Bread Board, LED)



Pin Name	VCP-G Board	GPIO
LED_Green(+)	6	A[13]

• Experiment circuit

- Requirement
 - Breadboard
 - LED * 1
- PDM 사용

VIM 사용법



- **VIM 에디터 사용법**

- VIM으로 파일 열기
 - `$ vim ~/test.c`
- 입력 : "i"
- 입력 취소 : "esc 키"
- 저장 후 종료 : ":wq"
- 지우기 : i(insert) 활성화 된 상태에서 "Backspace 키 혹은 Delete 키"

PDM Code

~/topst-vcp/sources/dev.drivers/pdm/pdm.c에 아래와 같이 명시

```
{ GPIO_PERICH_SEL_PWMSEL_3,    GPIO_GPA(13UL),    GPIO_FUNC(2UL),    GPIO_PERICH_CH0 },
```

GPIO_A13의 PDM을 사용할 것이기 때문에, pdm Channel은 3, FUNC는 2, port(gpio channel)는 GPIO_PERICH_CH0을 사용

PDM Code – pdm_led.h

~/topst-vcp/sources/app.sample/test.app.pdm/pdm_led.h

```
#ifndef MCU_BSP_PDM_LED_H
#define MCU_BSP_PDM_LED_H

void PDM_Led_Run(void); // main task에서 호출할 진입 함수

#endif
```

PDM Code – pdm_led.c

~/topst-vcp/sources/app.sample/test.app.pdm/pdm_led.c

```
#include <stdint.h>
#include <gpio.h>
#include <pdm.h>
#include "pdm_led.h"

static void ConfigureLedPWM(uint32 channel, uint32 port, uint32 duty_ns)
{
    PDMModeConfig_t pwm_cfg;
    uint32 wait_cnt = 0;

    pwm_cfg.mcPortNumber      = port; // 사용할 포트
    pwm_cfg.mcOperationMode   = PDM_OUTPUT_MODE_PHASE_1; // 기본 단일 위상 출력 모드
    pwm_cfg.mcInversedSignal  = 0; // 신호 반전 사용 안함
    pwm_cfg.mcOutSignalInIdle = 0; // 유휴 상태 시 출력 없음
    pwm_cfg.mcLoopCount       = 0; // 루프카운트(0:무한반복)
    pwm_cfg.mcOutputCt기     = 0; // 기본 출력 제어 설정
    pwm_cfg.mcPeriodNanoSec1  = 1000000; // 주기 1ms(1kHz)
    pwm_cfg.mcDutyNanoSec1    = duty_ns; // 듀티 설정 값
    pwm_cfg.mcPeriodNanoSec2  = 0; // 사용 x
    pwm_cfg.mcDutyNanoSec2    = 0; // 사용 x

    PDM_Disable(channel, PMM_ON); // PDM 비활성화
    while (PDM_GetChannelStatus(channel)) // PDM 꺼질 때까지 대기
    ... 다음 장에 계속
```

PDM Code – pdm_led.c

~/topst-vcv/sources/app.sample/test.app.pdm/pdm_led.c

```
.... 이어서
{
    SAL_TaskSleep(1);
    if (++wait_cnt > 100)
    {
        mcu_printf("Timeout on channel %d\n", channel);
        return;
    }
}

if (PDM_SetConfig(channel, &pwm_cfg) != SAL_RET_SUCCESS)
{
    mcu_printf("SetConfig fail (CH:%d)\n", channel);
    return;
}

if (PDM_Enable(channel, PMM_ON) != SAL_RET_SUCCESS)
{
    mcu_printf("Enable fail (CH:%d)\n", channel);
    return;
}
}
.... 다음 장에 계속
```

PDM Code – pdm_led.c

~/topst-vcv/sources/app.sample/test.app.pdm/pdm_led.c

```
.... 이어서
void PDM_Led_Run(void)
{
    PDM_Init(); // PDM 초기화
    ConfigureLedPWM(3, GPIO_PERICH_CH0, 0); // 초기 상태: LED off

    while (1)
    {
        for (uint32 duty = 0; duty <= 1000000; duty += 10000) // 밝기 증가 단계
        {
            ConfigureLedPWM(3, GPIO_PERICH_CH0, duty); // 듀티 증가
            SAL_TaskSleep(5);
        }

        for (uint32 duty = 1000000; duty > 0; duty -= 10000) // 밝기 감소 단계
        {
            ConfigureLedPWM(3, GPIO_PERICH_CH0, duty); // 듀티 감소
            SAL_TaskSleep(5);
        }
    }
}
```

PDM Code – rules.mk / main.c

~/topst-vcv/sources/app.sample/test.app.pdm/rules.mk ~/topst-vcv/sources/app.sample/app.base/main.c

```
SRCS += pdm_test.c
// 추가
SRCS += pdm_led.c
```

```
#include "pdm_led.h" // 변경

static void Main_StartTask(void * pArg)
{
    (void)pArg;
    (void)SAL_OsInitFuncs();

    // 변경
    PDM_Led_Run();
}
```

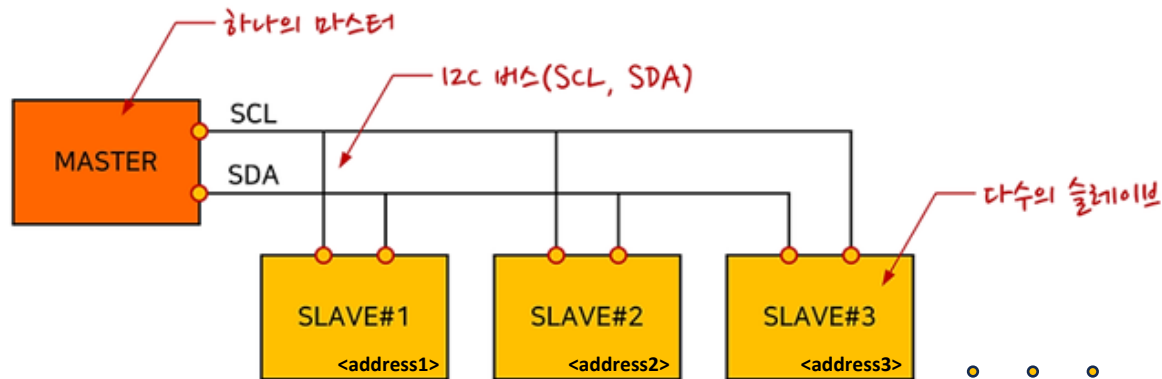

Build

- build 진행
 - `$ cd ~/topst-vcp/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/topst-vcp/tools/fwdn_vcp/fwdn`
`--fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/topst-vcp/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- 보드 재부팅 시 LED fade in/out 반복.

I2C 이해

• I2C(Inter-Integrated Circuit)

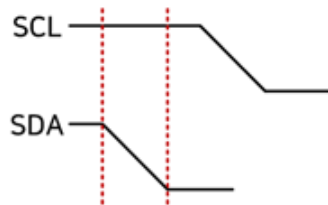
- SDA, SCL 신호선으로 다수의 I2C 통신 지원
- 마스터에서 기준 클럭(SCL) 생성, 클럭에 맞춰 데이터(SDA) 송수신
- 하나의 마스터와 다수의 슬레이브로 연결이 구성됨



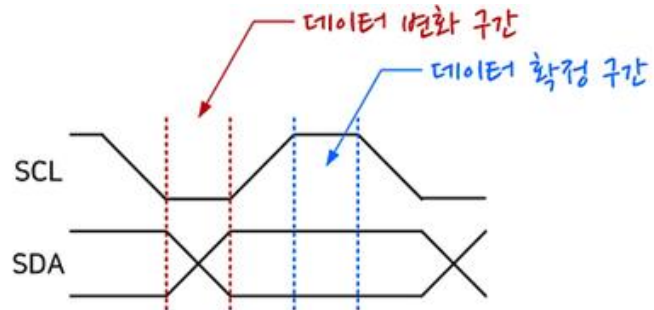
- 반이중 통신 (송수신 동시 x)

I2C 이해

- SCL = HIGH 유지, SDA = HIGH -> LOW 면 Start



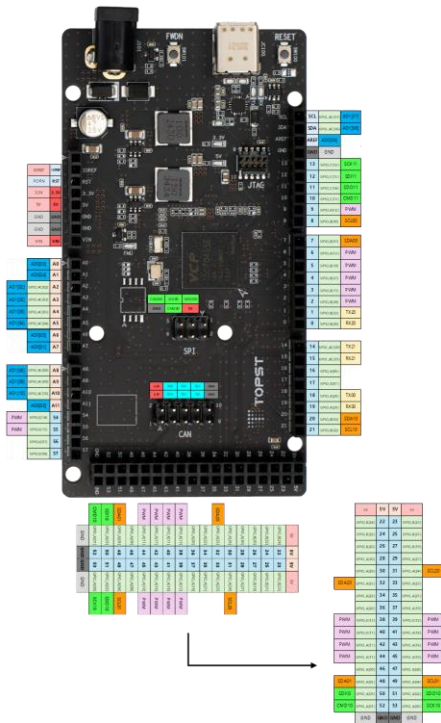
- SCL = LOW -> 데이터 바꿈, SCL = HIGH -> 데이터 확정



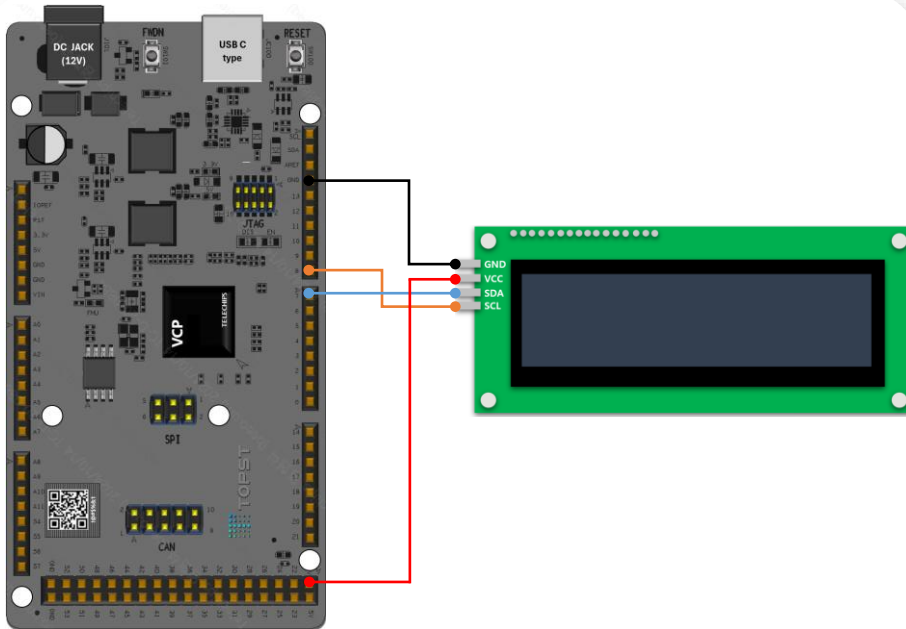
- I2C 센서 : LCD, Humidity/Temp, Accelerometer ...

VCP-G 핀맵

- TOPST DOCS에서 VCP-G 핀 FUNC 및 GPIO 번호 확인
 - <https://topst.ai/tech/docs?page=1714a3dcdf19dbf61088d7364d61e29f90a15ff2>



회로 구성 (VCP-G ↔ Bread Board, LCD)



Pin Name	VCP-G Board	GPIO
LCD 1602 VCC	5V	-
LCD 1602 GND	GND	-
LCD 1602 SDA	7	B[26]
LCD 1602 SCL	8	B[25]

• Experiment circuit

- Requirement
 - Breadboard
 - LCD 1602 * 1
- I2C 사용

VIM 사용법



- **VIM 에디터 사용법**

- VIM으로 파일 열기
 - `$ vim ~/test.c`
- 입력 : "i"
- 입력 취소 : "esc 키"
- 저장 후 종료 : ":wq"
- 지우기 : i(insert) 활성화 된 상태에서 "Backspace 키 혹은 Delete 키"

I2C Code

~/topst-vcp/sources/dev.driverse/i2c/i2c.c에 아래와 같이 명시

```
{ 0UL, GPIO_GPB(0UL), GPIO_GPB(1UL), GPIO_FUNC(1UL), GPIO_PERICH_CH0 }, // I2C0, CH_0
```

사용할 I2C 채널 및 포트가 0이기 때문에 0으로 설정

~/topst-vcp/sources/dev.driverse/i2c/lcd.h에 아래와 같이 명시되어 있음

```
#define I2C_CH      0
#define I2C_PORT    0
#define LCD_ADDR    0x27
#define I2C_SPEED   100
#define LCD_CMD     0
#define LCD_DATA    1
```

I2C Code – lcd1602.h / lcd1602.c

~/topst-vcp/sources/app.sample/test.app.i2c/lcd1602.h ~/topst-vcp/sources/app.sample/test.app.i2c/lcd1602.c

```
#ifndef MCU_BSP_LCD1602_H
#define MCU_BSP_LCD1602_H

void LCD1602_Run(void); // main task에서 호출할 진입 함수

#endif
```

```
#include <i2c.h>
#include <lcd.h>
#include "lcd1602.h"

void LCD1602_Run(void)
{
    SAL_OsInitFuncs();

    I2C_Init(); // I2C 초기화
    if (I2C_Open(I2C_CH, I2C_PORT, I2C_SPEED, NULL,
    NULL) != SAL_RET_SUCCESS) {
        mcu_printf("I2C open failed\n");
        return; //오픈 실패 시 진행하지 않고 fail 로그
    }

    uint32 detected_addr = I2C_ScanSlave(I2C_CH); //스캔
    mcu_printf("Detected I2C Slave Address: 0x%02X\n",
    detected_addr);

    lcd_init(); // LCD 초기화
    lcd_cmd(0x80); // 커서를 맨 앞으로
    lcd_print("Hello TOPST");

    while (1) {
        SAL_TaskSleep(1000);
    }
}
```


I2C Code – rules.mk / main.c

~/topst-vcp/sources/app.sample/test.app.i2c/rules.mk ~/topst-vcp/sources/app.sample/app.base/main.c

```
# Paths
VPATH += $(MCU_BSP_APP_SAMPLE_I2C_TEST_PATH)
// 추가
VPATH += $(MCU_BSP_DEV_DRIVERS_I2C_PATH)

# Includes
INCLUDES += -I$(MCU_BSP_APP_SAMPLE_I2C_TEST_PATH)
// 추가
X

SRCS += i2c_test.c
// 추가
SRCS += lcd1602.c
SRCS += lcd.c
```

```
#include "lcd1602.h" // 변경

Static void Main_StartTask(void * pArg)
{
    (void)pArg;
    (void)SAL_OsInitFuncs();

    // 변경
    LCD1602_Run();
}
```

Build

- build 진행
 - `$ cd ~/topst-vcp/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/topst-vcp/tools/fwdn_vcp/fwdn`
`--fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/topst-vcp/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- 보드 재부팅 시 LCD에 "Hello TOPST" 문구 출력. (안보일 시 LCD판 뒤의 가변저항 조절)
- `sudo minicom -D /dev/ttyUSB0 -b 115200 -8` 실행 후 보드 재부팅 시 UART 로그에 I2C Channel출력.

VCP-G 환경 구성

```
ead@TC-D-20-0546:~/test$ git clone https://github.com/topst-development/FreeRTOS-VCP
Cloning into 'FreeRTOS-VCP'...
remote: Enumerating objects: 1430, done.
remote: Counting objects: 100% (183/183), done.
remote: Compressing objects: 100% (166/166), done.
remote: Total 1430 (delta 30), reused 146 (delta 15), pack-reused 1247 (from 2)
Receiving objects: 100% (1430/1430), 90.07 MiB | 4.03 MiB/s, done.
Resolving deltas: 100% (425/425), done.
ead@TC-D-20-0546:~/test$ cd FreeRTOS-VCP/
ead@TC-D-20-0546:~/test/FreeRTOS-VCP$ ls
build  easy-setup_vcp-g.sh  LICENSE  scripts  sources  tools
```

- **Clone Source Code**

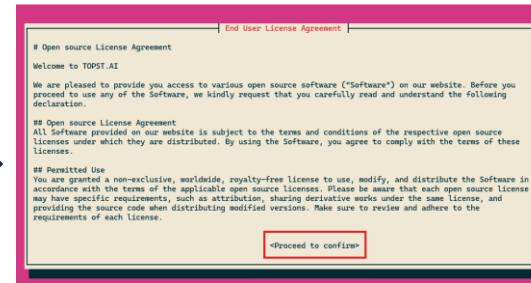
- WSL2에서 VCP-G FreeRTOS 소스 코드를 가져오기 위해 git clone
 - \$ cd
 - \$ git clone https://github.com/topst-development/FreeRTOS-VCP
- 소스 코드가 포함된 디렉터리로 이동 후 확인
 - \$ cd FreeRTOS-VCP
 - \$ git checkout edu/3_day_short_course

VCP-G 환경 구성

```
build easy-setup_vcp-g.sh LICENSE scripts sources tools
ead@TC-D-20-0546:~/test/FreeRTOS-VCP$ ./easy-setup_vcp-g.sh
cd build/tcc70xx/gcc/
make
ead@TC-D-20-0546:~/test/FreeRTOS-VCP$ cd build/tcc70xx/gcc/
```

• Build Guide

- Execute easy-setup_vcp-g.sh
 - \$./easy-setup_vcp-g.sh
- 라이선스 [Tab] -> [Enter]로 동의 후 build 진행
 - \$ cd build/tcc70xx/gcc
 - \$ make
- Build 후 생성된 output 디렉터리에서 ROM 파일 확인 가능
 - \$ ls -al output/

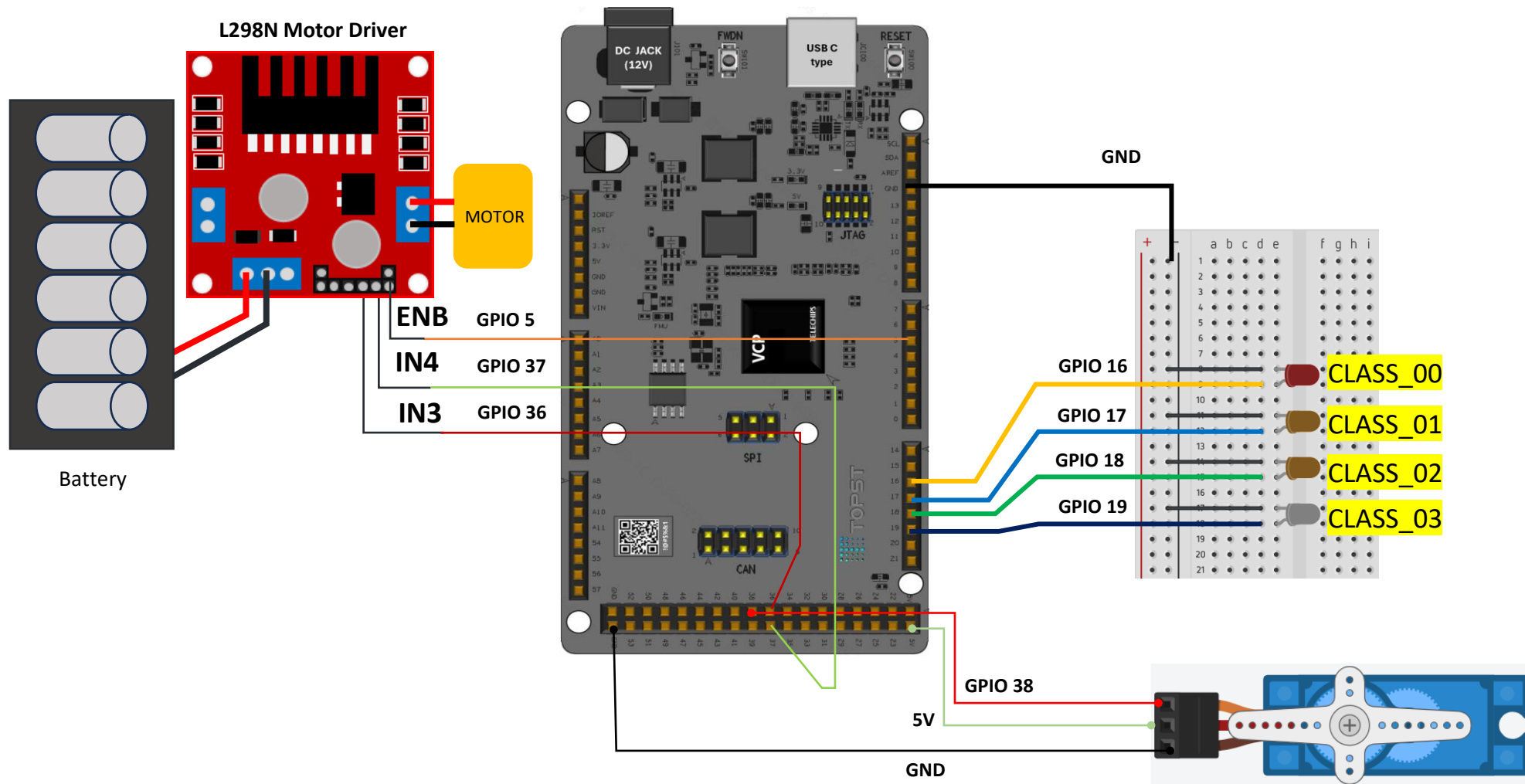


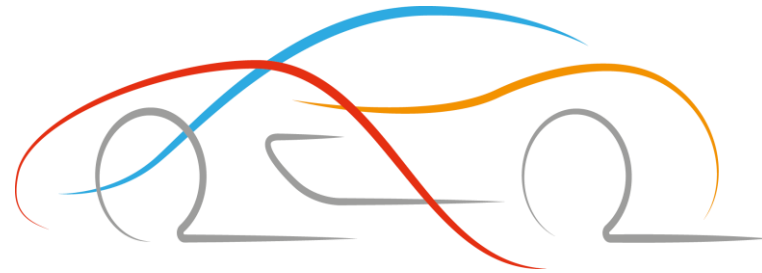
VCP-G Build

- Build 완료 후 FWDN (빌드한 디렉토리어서 바로 실행)
 - `$ sudo ~/FreeRTOS-VCP/tools/fwdn_vcp/fwdn --fwdn ~/FreeRTOS-VCP/tools/fwdn_vcp/vcp_fwdn.rom -w ~/FreeRTOS-VCP/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- `minicom -D /dev/ttyUSB0 -b 115200 -8` 실행
- 보드 리셋 진행

VCP-G 와 모형차 Kit 회로 연결

VCP-G





Thank You